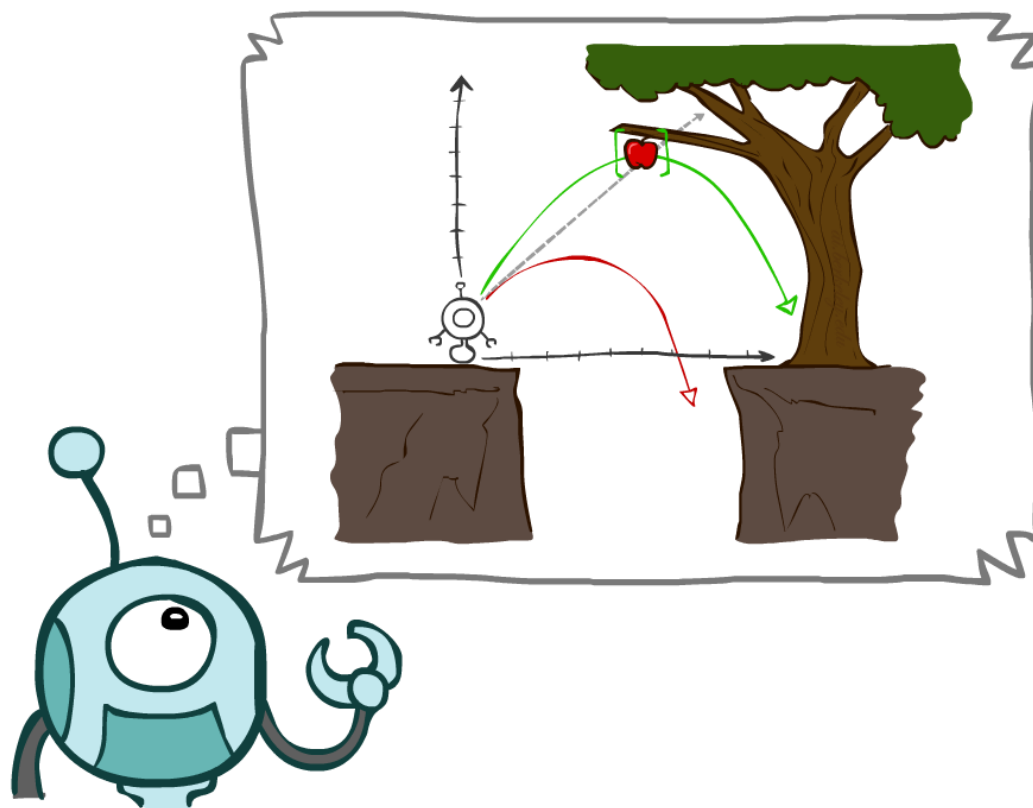




第二章：搜索

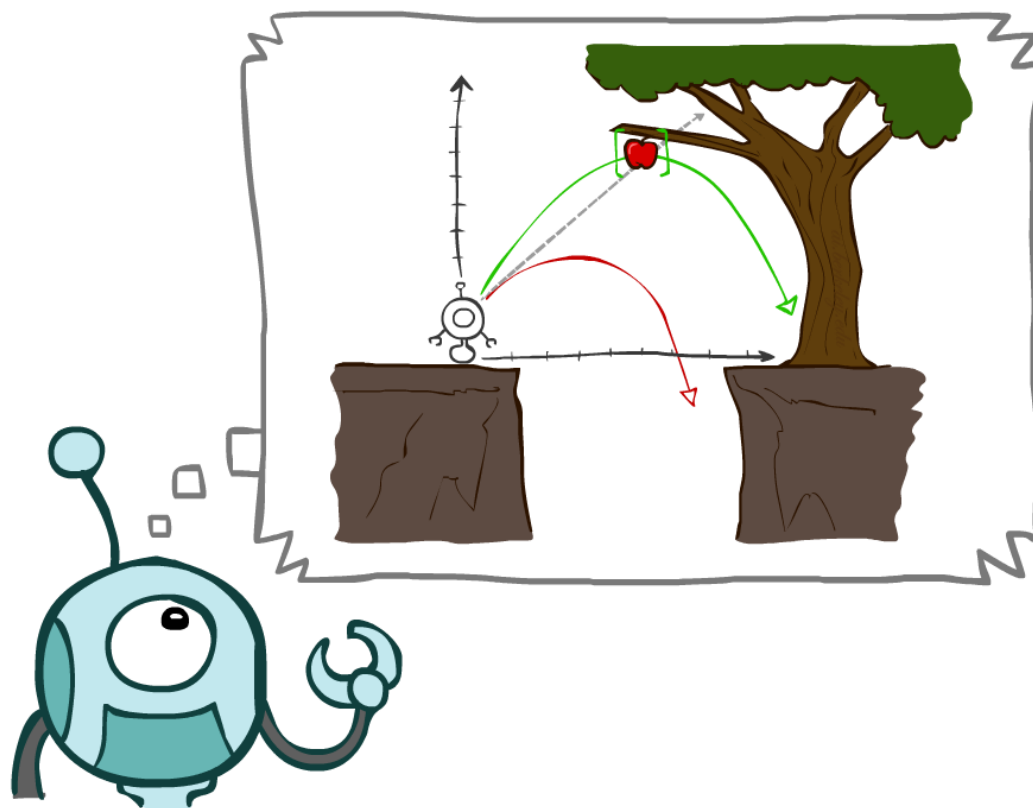
目录

- 搜索算法基础
- 无信息搜索（盲目搜索）
- 有信息搜索（启发式搜索）
- 局部搜索
- 约束满足问题

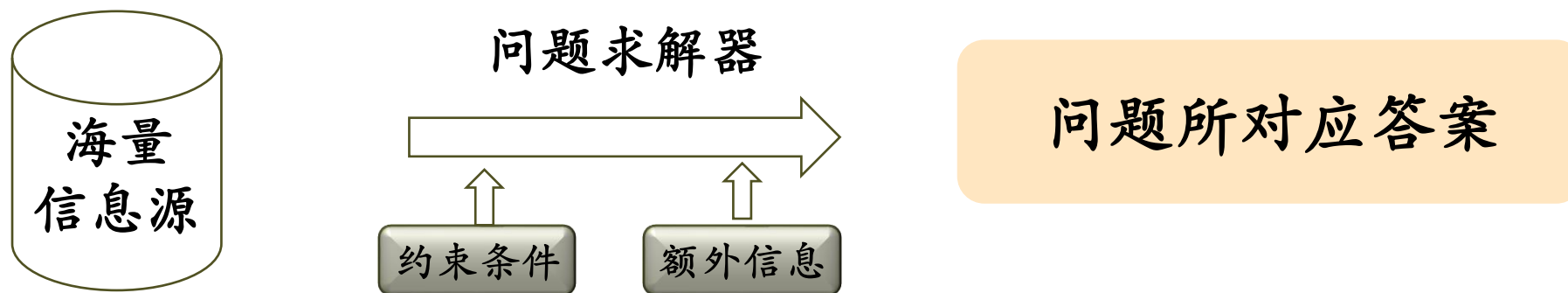


目录

- 搜索算法基础
- 无信息搜索（盲目搜索）
- 有信息搜索（启发式搜索）
- 局部搜索
- 约束满足问题



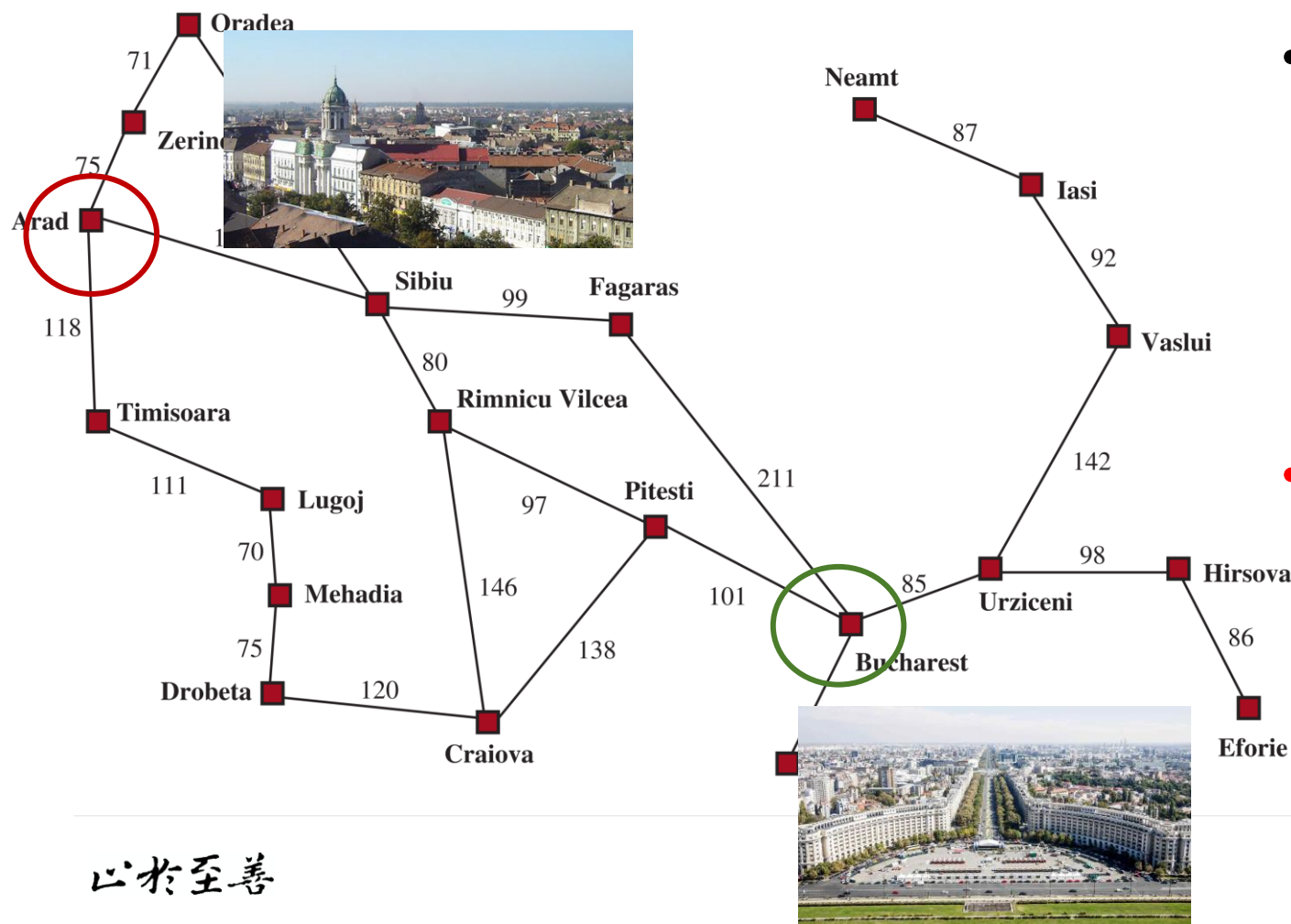
人工智能中的搜索



搜索：当采取何种正确行动并不立即显而易见时，考虑一系列形成通向目标状态路径的动作 —— 其执行的计算过程被称为搜索

搜索算法的形式化描述

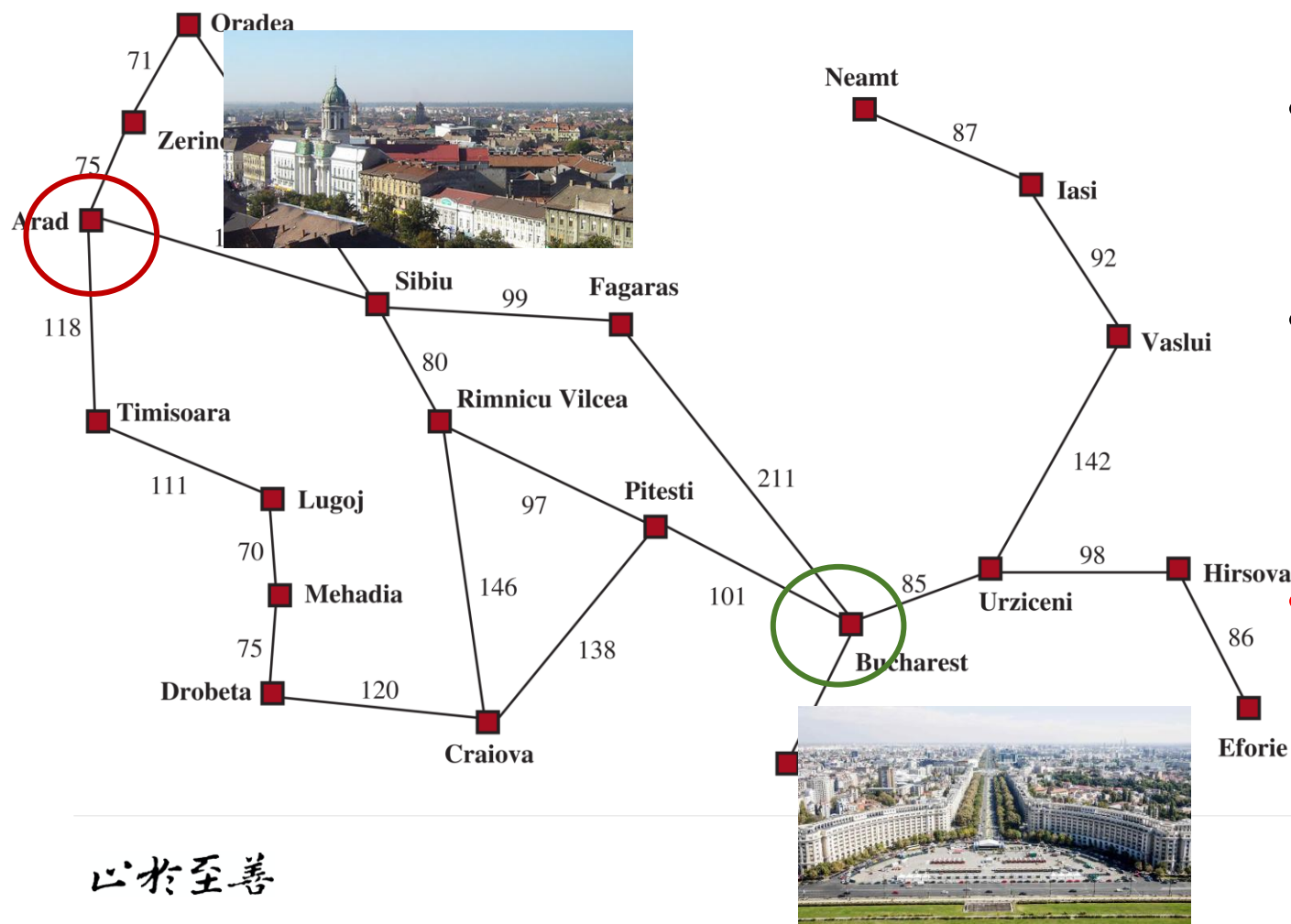
<状态、后继函数（动作+代价）、状态转移、路径、目标测试>



- 状态：对智能体和环境当前情形的描述。例如，在最短路径问题中，城市可作为状态。将原问题城市对应的状态称为**初始状态**。**状态空间**包含了所有的可能状态
- **后继函数**（动作+代价）：状态通过动作选择而产生连接的关系。动作表示从当前时刻所处状态转移到下一时刻所处状态所进行操作。一般而言这些操作都是离散的，且有代价

搜索算法的形式化描述

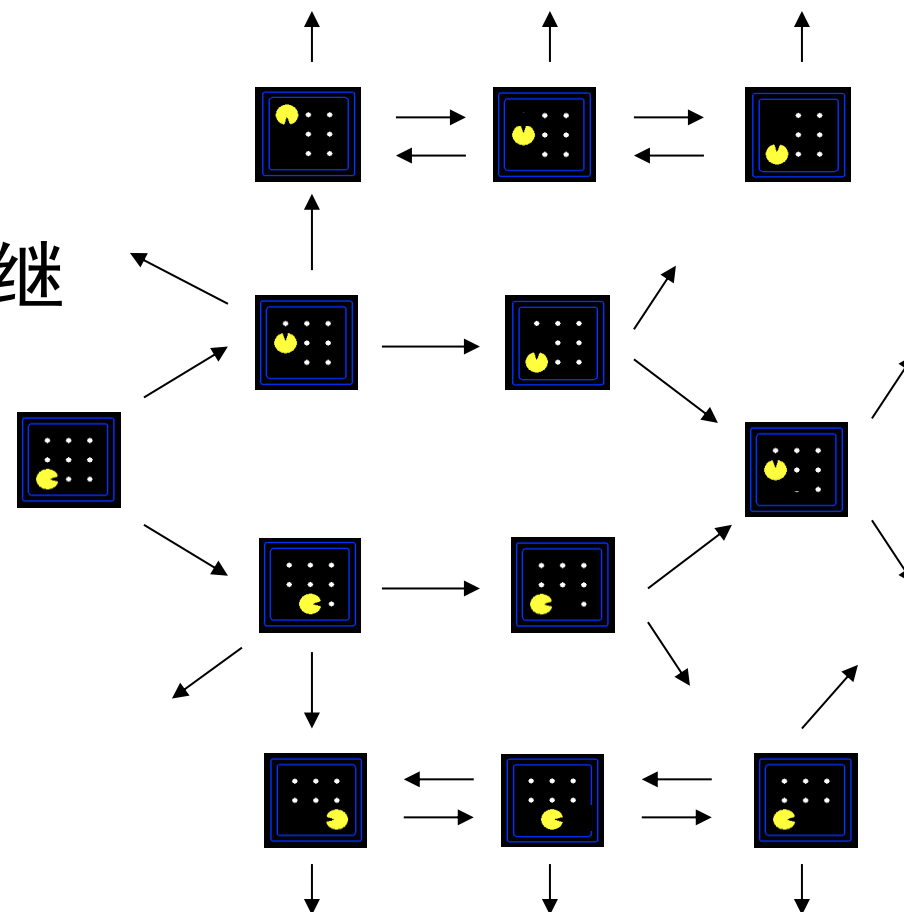
〈状态、后继函数（动作+代价）、状态转移、路径、目标测试〉



- 状态转移：智能体选择了一个动作之后，其所处状态的相应变化
- 路径：一个状态序列。该状态序列被一系列操作所连接。如从A到B所形成的路径
- **目标测试**：评估当前状态是否为所求解的目标状态

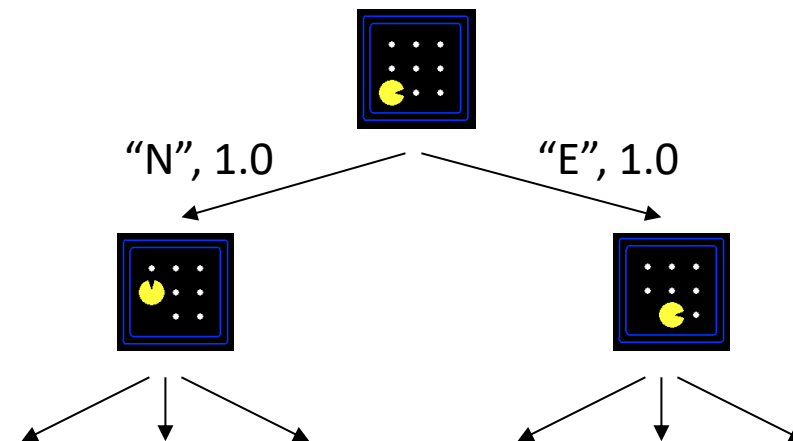
建模搜索问题：状态空间图

- 状态空间图：建模搜索问题的第一种方式
- 空间图中节点与状态一一对应，边代表后继函数
- 目标测试是一组/一个目标节点
- 在状态空间图中，每个状态只出现一次！
- 图中可能存在环路



建模搜索问题：搜索树

- 搜索树：建模搜索问题的第二种方式
- 初始状态是根节点
- 子节点对应父节点的后继状态
- 每个节点除了对应到一个状态之外还标志了一条从初始状态到当前状态的动作序列
- 实现相同状态的不同路径在树中是不同的节点->状态和节点不是一对一的



搜索算法框架：树搜索

函数：TreeSearch

输入：节点选择函数 `pick_from`，后继节点计算函数 `successor_nodes`

输出：从初始状态到终止状态的路径

```
1  $\mathcal{F} \leftarrow \{\text{根节点}\}$     从搜索树的根节点开始
2 while  $\mathcal{F} \neq \emptyset$  do    边缘：已被生成（发现）但未被访问的候选节点（数据结构）
3      $n \leftarrow \text{pick\_from}(\mathcal{F})$   根据搜索策略，在边缘中确定下一个要访问的节点
4      $\mathcal{F} \leftarrow \mathcal{F} - \{n\}$     访问下一个节点
5     if goal_test(n) then 判断当前节点对应的状态是不是目标状态
6         return  $n.\text{path}$ 
7     end
8      $\mathcal{F} \leftarrow \mathcal{F} \cup \text{successor\_nodes}(n)$  生成这个节点的所有后继节点，将新节点加入边缘
9 end
```

搜索算法的评价指标

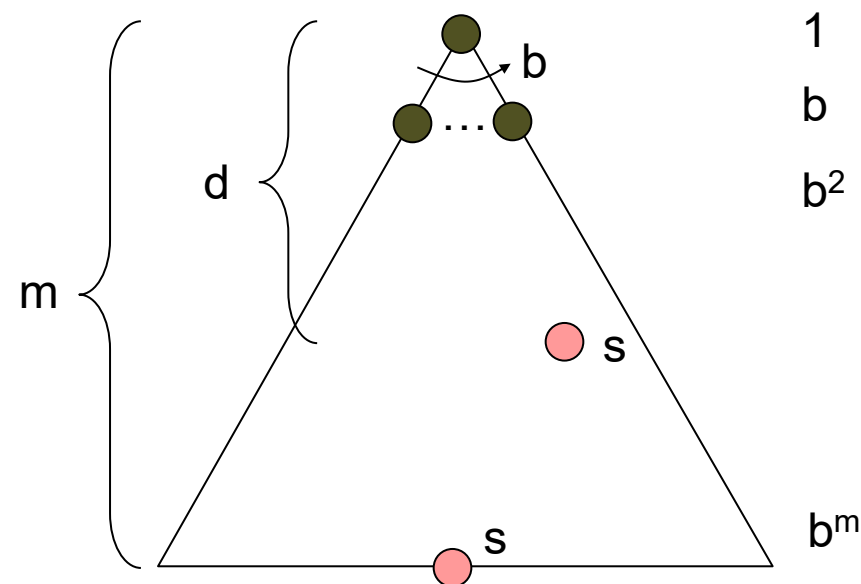
完备性	当问题存在解时，算法是否能保证找到一个解。
最优性	搜索算法是否能保证找到开销最小的解。
时间复杂度	找到一个解所需时间（生成节点的数量）。
空间复杂度	算法运行过程消耗的内存量（储存节点的数量）。

完备性和最优性刻画了算法找到解的能力以及所求的解的质量，时间复杂度和空间复杂度衡量了算法的资源消耗，它们通常用 O 符号来描述

搜索算法的评价指标

搜索树中用于估计复杂度的变量含义

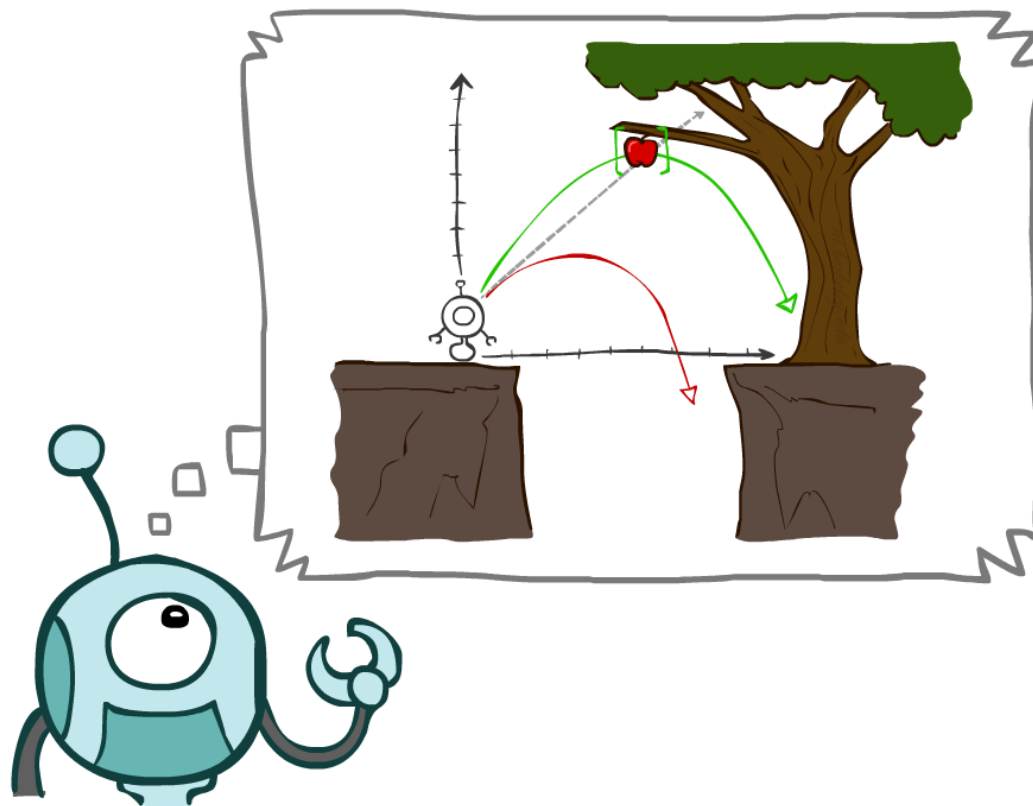
符号	含义
b	分支因子，即搜索树中每个节点最大的分支数目
d	根节点到最浅的目标结点的路径长度
m	搜索树中路径的最大可能长度



树中节点数: $1 + b + b^2 + \dots + b^m = O(b^m)$

目录

- 搜索算法基础
- 无信息搜索（盲目搜索）
- 有信息搜索（启发式搜索）
- 局部搜索
- 约束满足问题



无信息搜索策略只使用问题定义中的信息，不提供有关某个状态与目标状态的接近程度的任何线索

- Breadth-first search 广度优先搜索
- Uniform-Cost Search 一致代价搜索
- Depth-first search 深度有效搜索
- Depth-limited search 深度受限搜索
- Iterative deepening search 迭代加深搜索

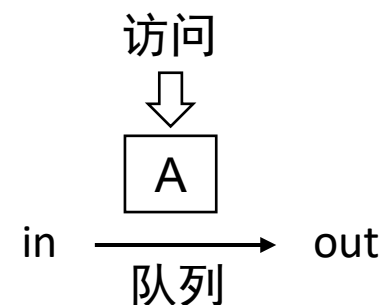
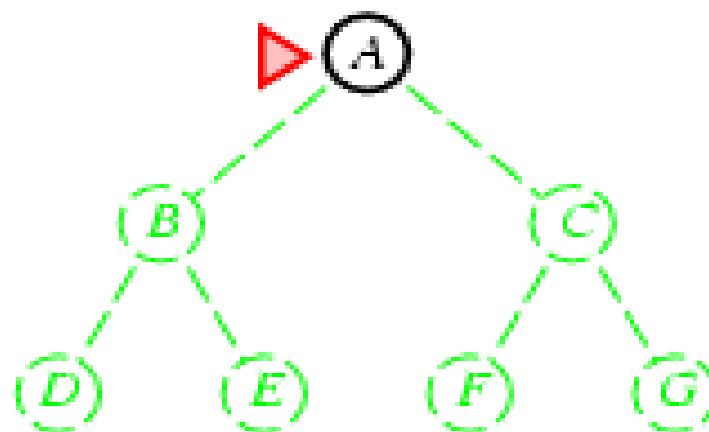
广度优先搜索



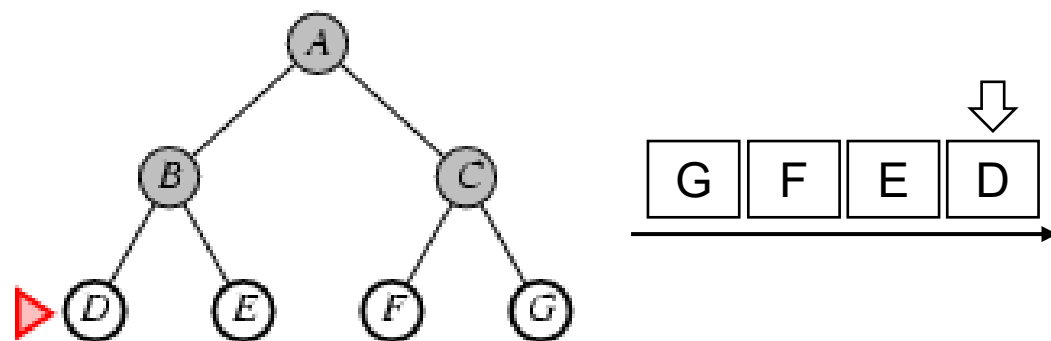
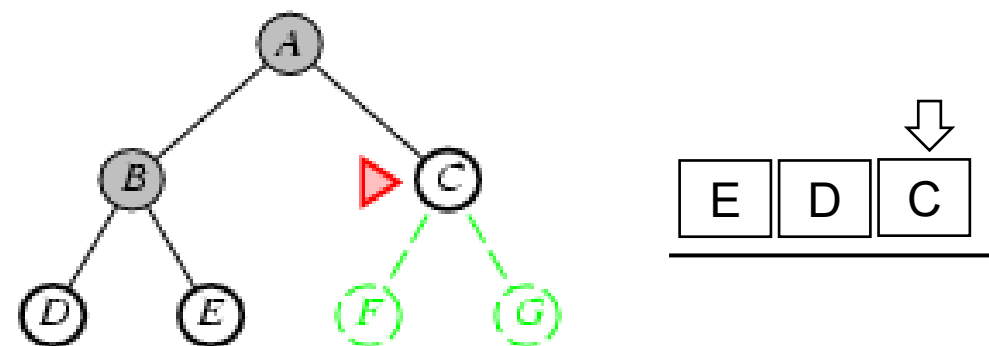
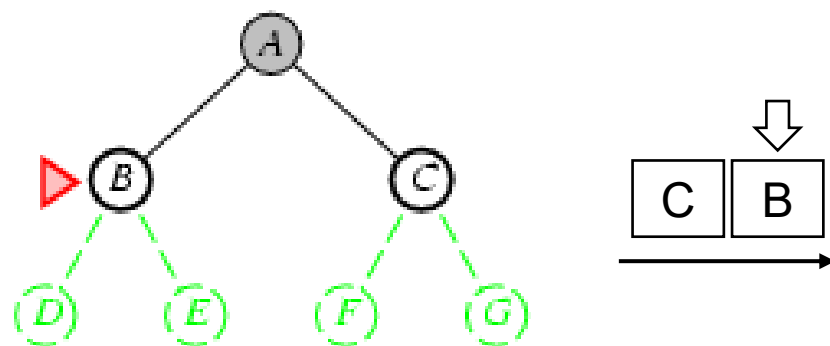
广度优先搜索

访问边缘中最浅层的节点

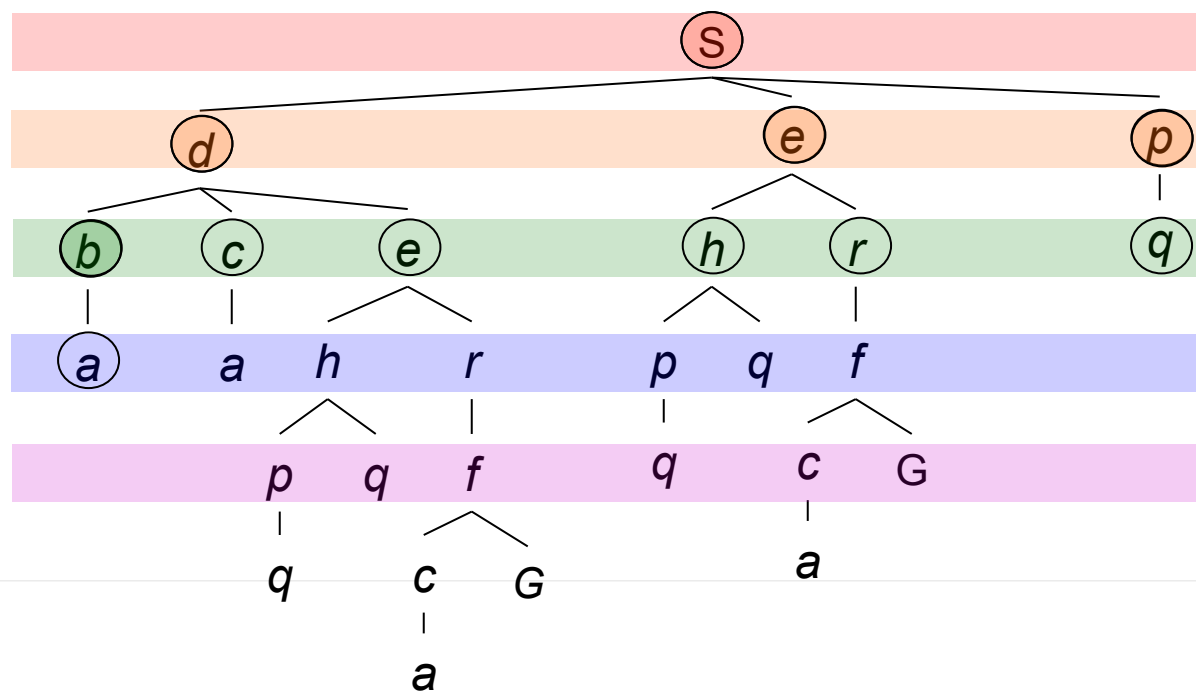
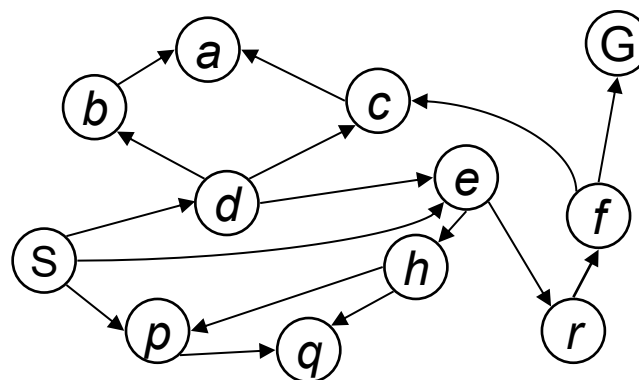
实现方法：边缘是先进先出队列



广度优先搜索



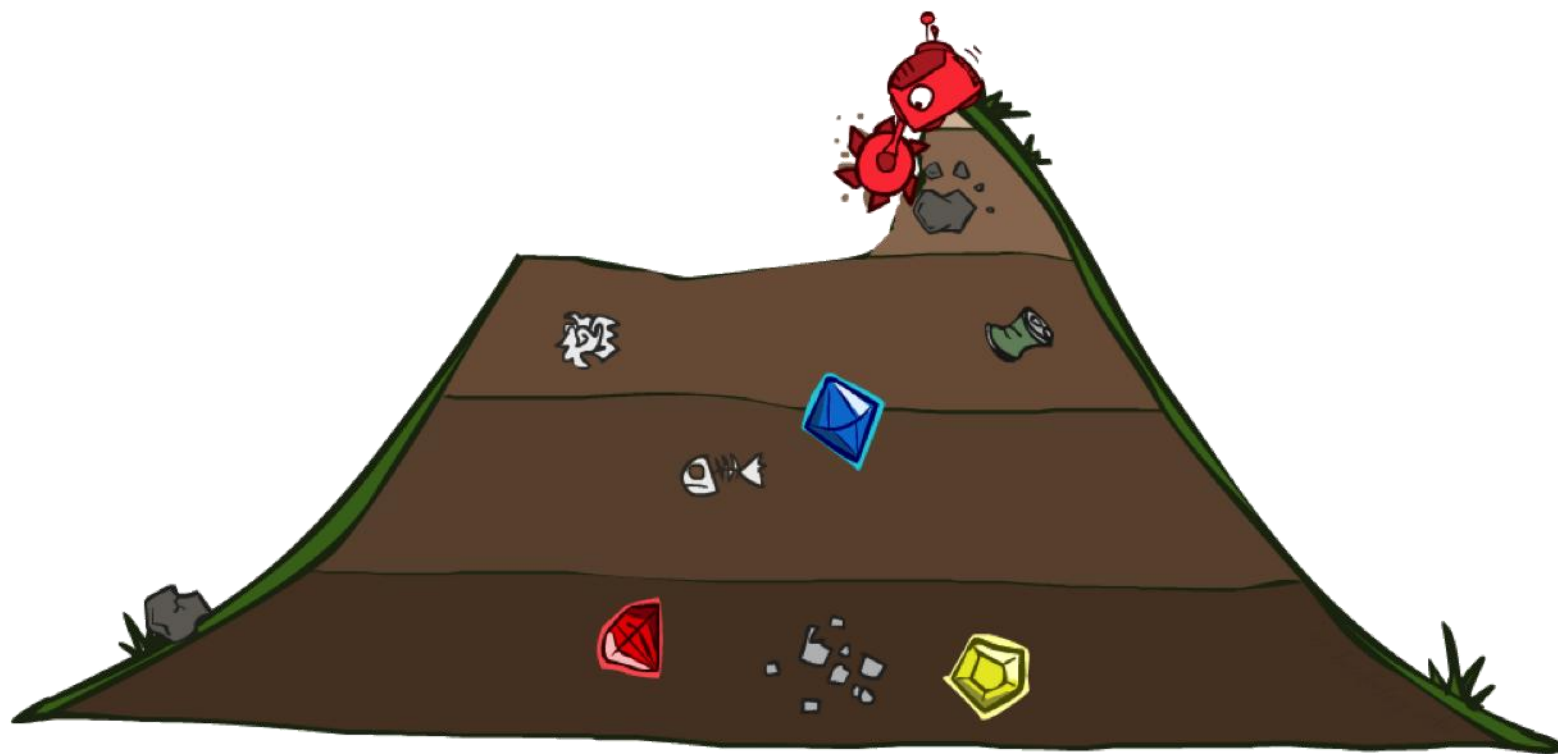
广度优先搜索



广度优先搜索性质

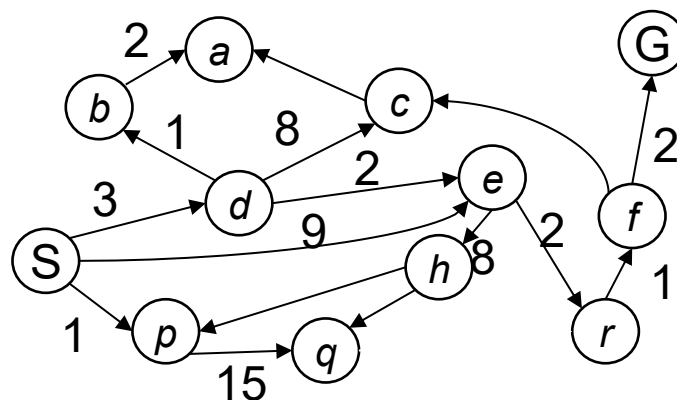
- 时间复杂度（生成节点的个数）假设解的深度是 d ，则
 $1+b+b^2+b^3+\dots+b^{d+1} = O(b^{d+1})$
- 空间复杂度（存储节点的个数） $O(b^{d+1})$
- 完备性 是（如果分支因子有限）
- 最优性 是（如果路径代价是节点深度的非递减函数）

一致代价搜索 (Dijkstra)

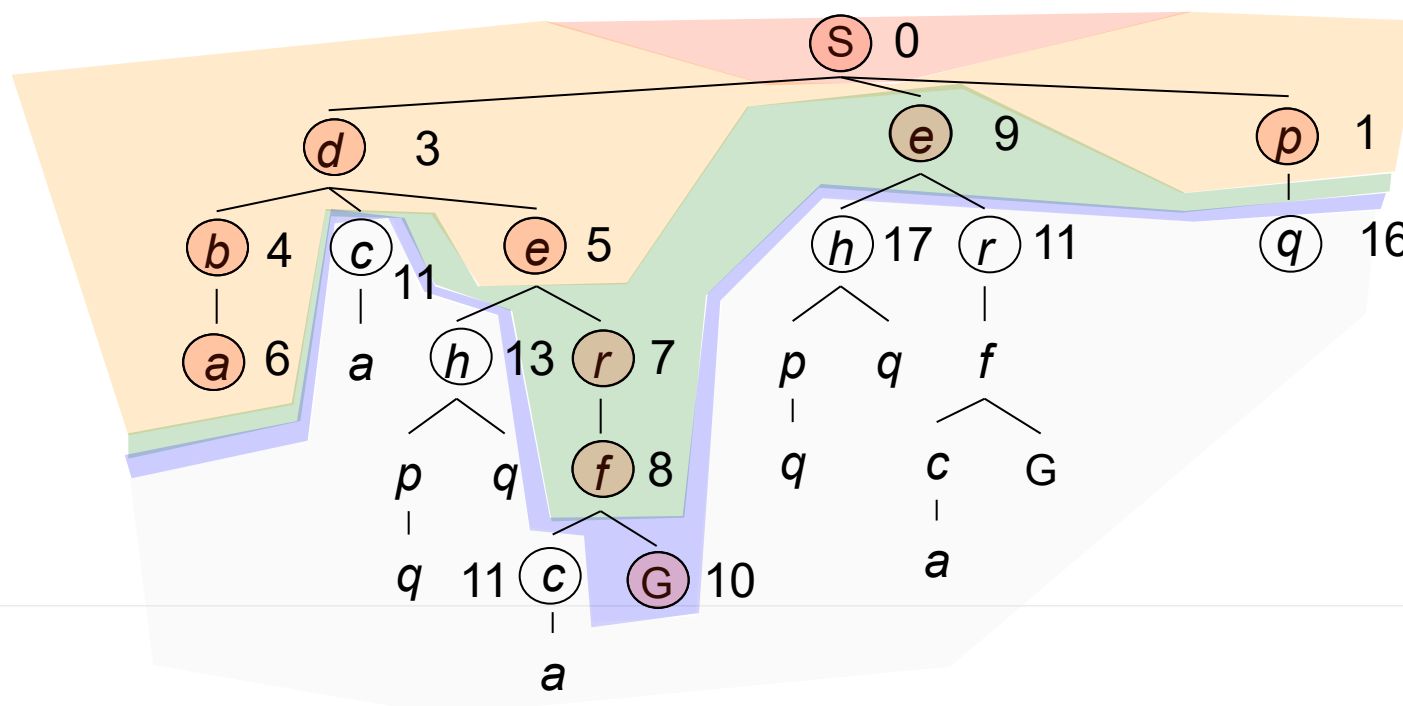
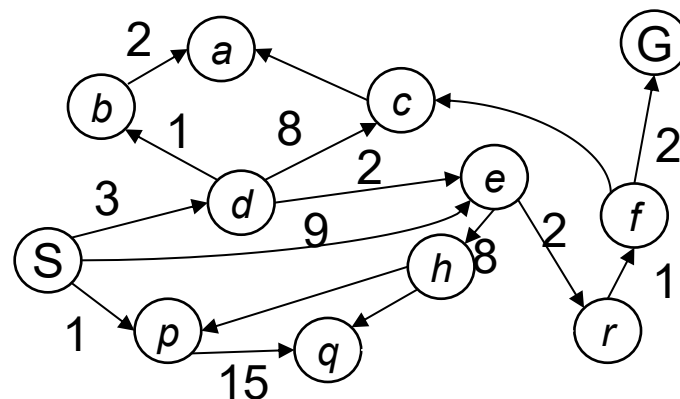


一致代价搜索

访问边缘中最“便宜”的节点
实现方法：边缘是优先队列



一致代价搜索



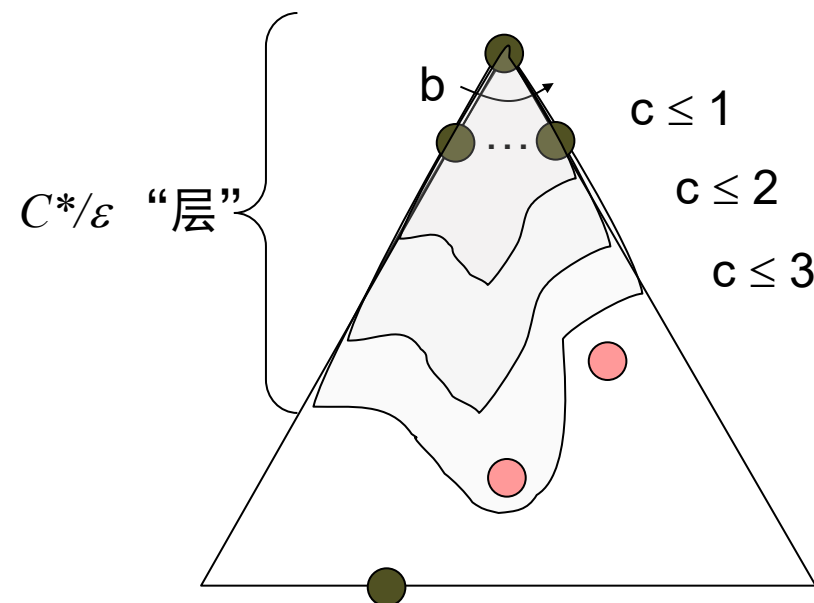
一致代价搜索性质

- 时间复杂度（生成节点的个数）

如果最优节点累计代价是 C^* ，最小单步代价是 ε ，那么从根节点到最优节点的最大步数 C^*/ε ，则 $O(b^{C^*/\varepsilon+1})$

- 空间复杂度（存储节点的个数） $O(b^{C^*/\varepsilon+1})$

- 完备性 是（如果最优节点代价有穷，最小单步代价大于0）
- 最优性 是



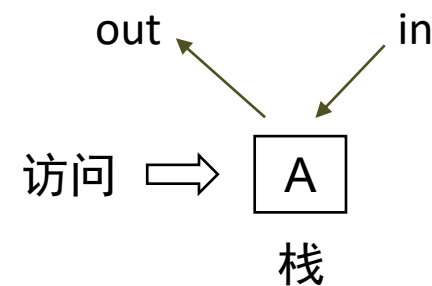
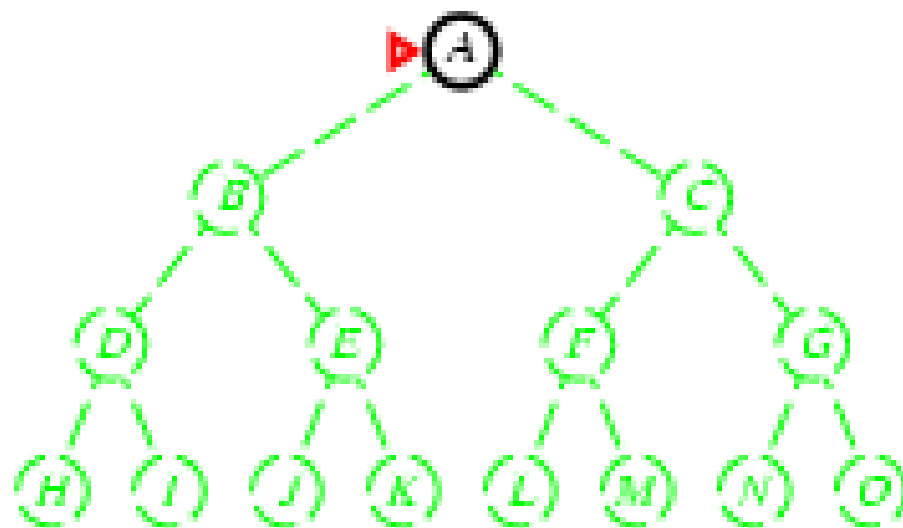
深度优先搜索



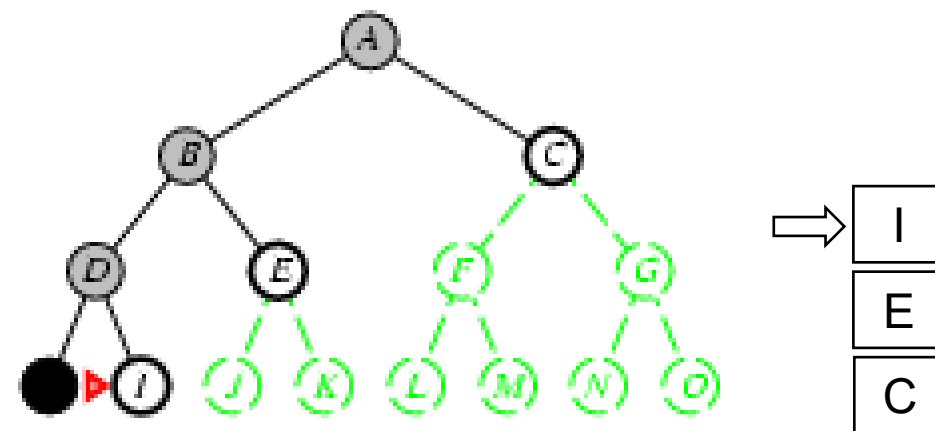
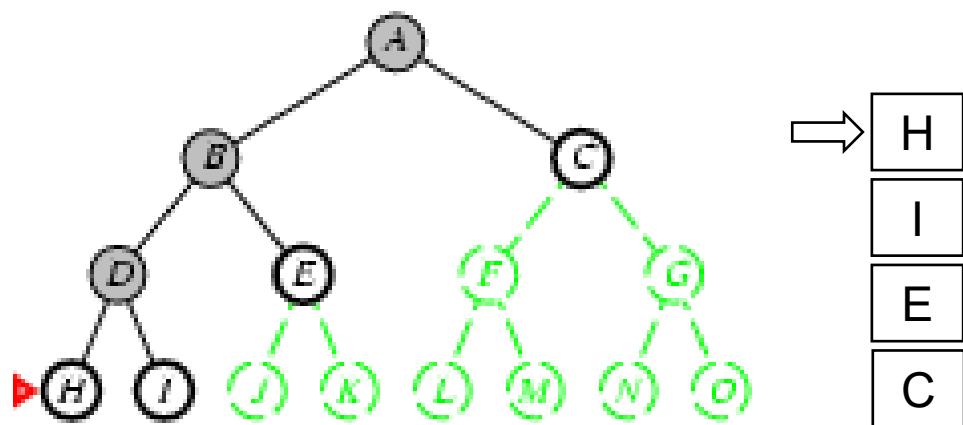
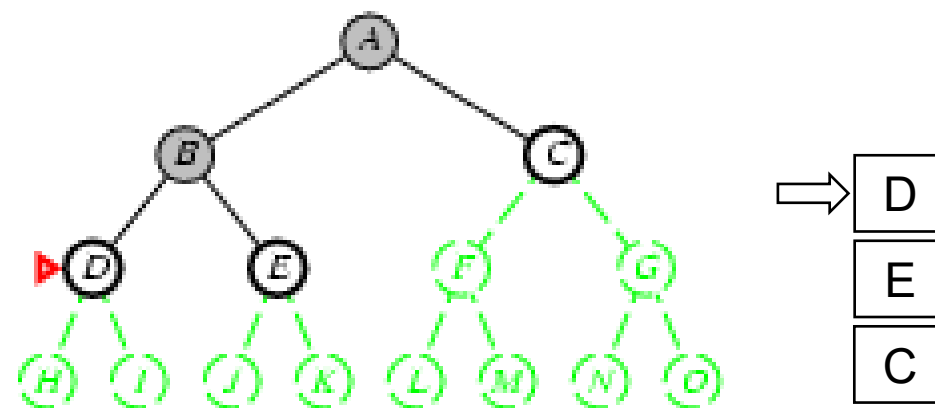
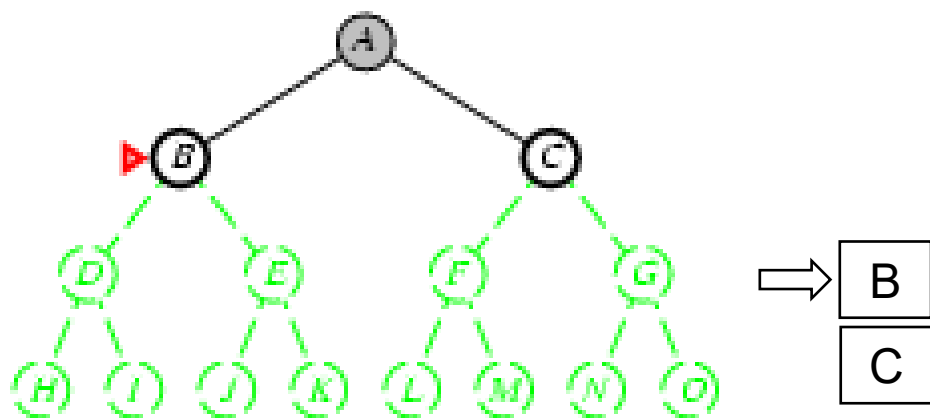
深度优先搜索

访问边缘中**最深层**的节点

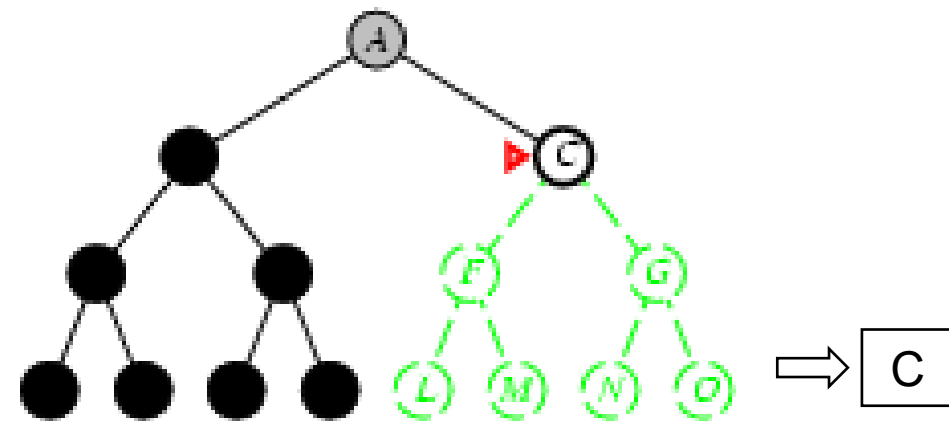
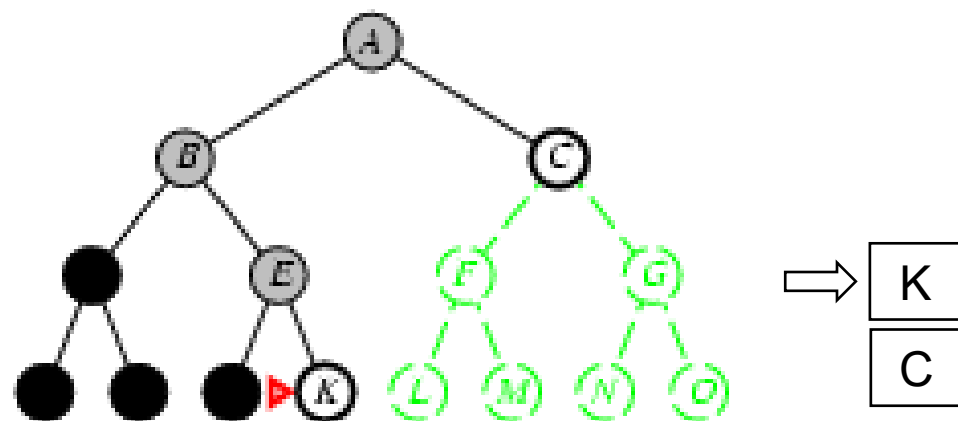
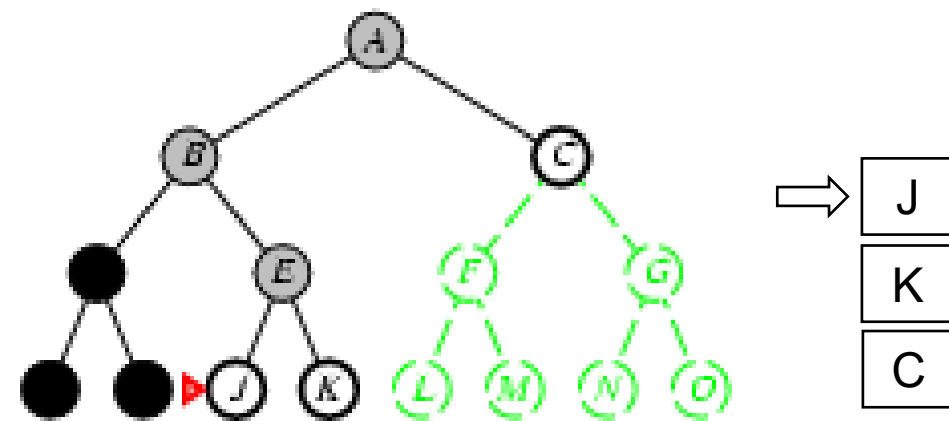
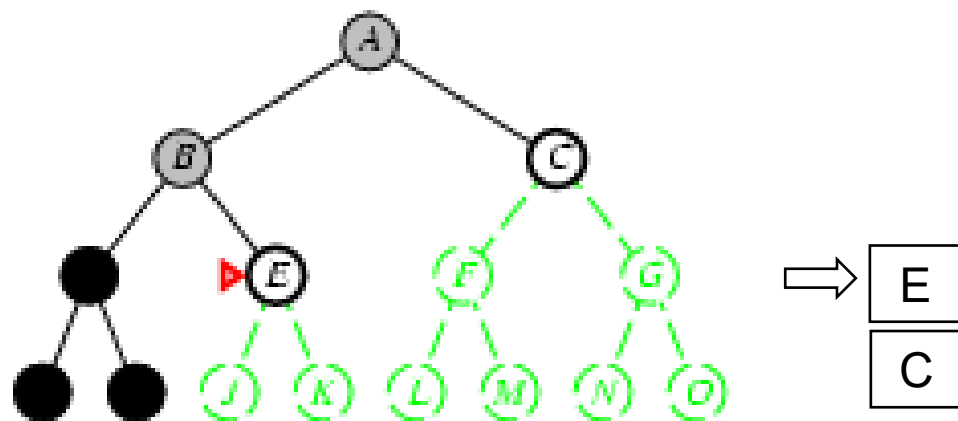
实现方法：边缘是先进后出队列（栈）

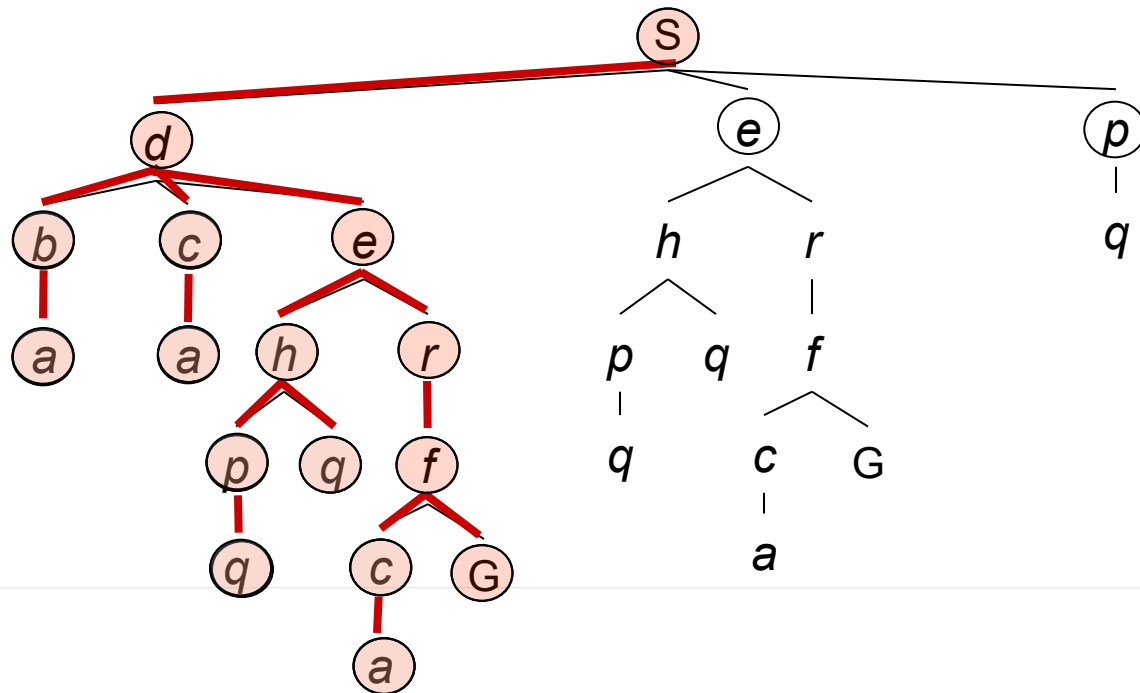
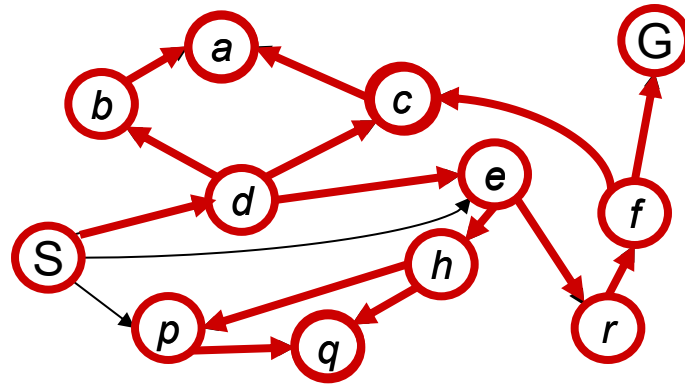


深度优先搜索



深度优先搜索





深度优先搜索性质



- 时间复杂度（生成节点的个数） 如果最大树高 m 是有限的，则 $O(b^m)$
- 空间复杂度（存储节点的个数） $O(bm)$
- 完备性 否（如果状态空间图中存在环，树的高度无限）
- 最优性 否

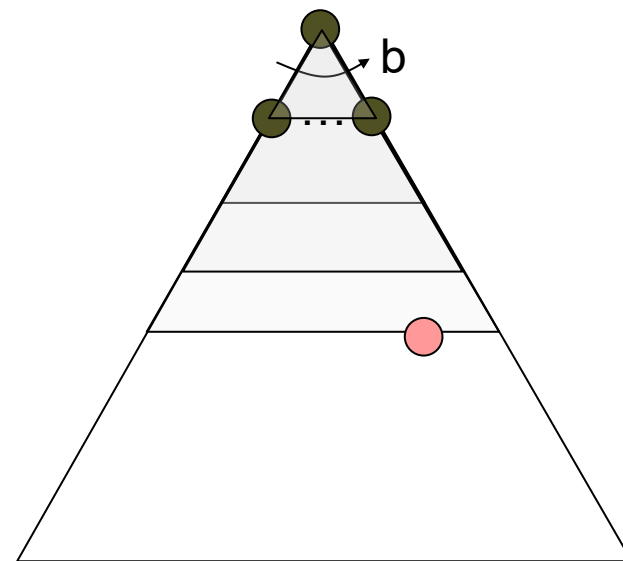
深度受限搜索

- 认为 l 层的节点没有后继节点 $\rightarrow$ 解决无限循环问题
- 时间复杂度 $O(b^l)$
- 空间复杂度 $O(bl)$
- 完备性 否（如果 $l < s$ ）
- 最优性 否（如果 $l < s$ ）

- 当搜索引擎用爬虫去抓取网页时，本质上就是在做图遍历。互联网公司在抓取网页时，绝大多数情况采用类广度优先策略，因为网站的首页（深度浅）通常比深层的分页更重要、质量更高
- 如果你在领英搜“张三”，系统要告诉你他是你的“二度人脉”，需要用广度优先，因为它能保证找到的是最短路径
- 这是目前最前沿的联系。在让大模型解决复杂逻辑问题时，研究者会使用 ToT (Tree of Thoughts) 框架：模型生成一个思路，如果发现不对，就回溯。为了防止模型在某个错误思路产生太长的幻觉），可以采用类似“迭代加深”的策略，限制每一轮思考的深度，逐步引导模型深入

迭代加深搜索

- 如何结合广度优先搜索的时间优势和深度优先搜索的空间优势？
- 迭代加深搜索是一个用来寻找最合适的深度限制的通用策略：不断增加深度限制，直到找到目标节点
- 当深度限制达到最浅目标节点的深度 d ，就找到了目标节点
 - ✓ Run a DFS with depth limit 1. If no solution...
 - ✓ Run a DFS with depth limit 2. If no solution...
 - ✓ Run a DFS with depth limit 3.



迭代加深搜索 $l = 0$

Limit = 0



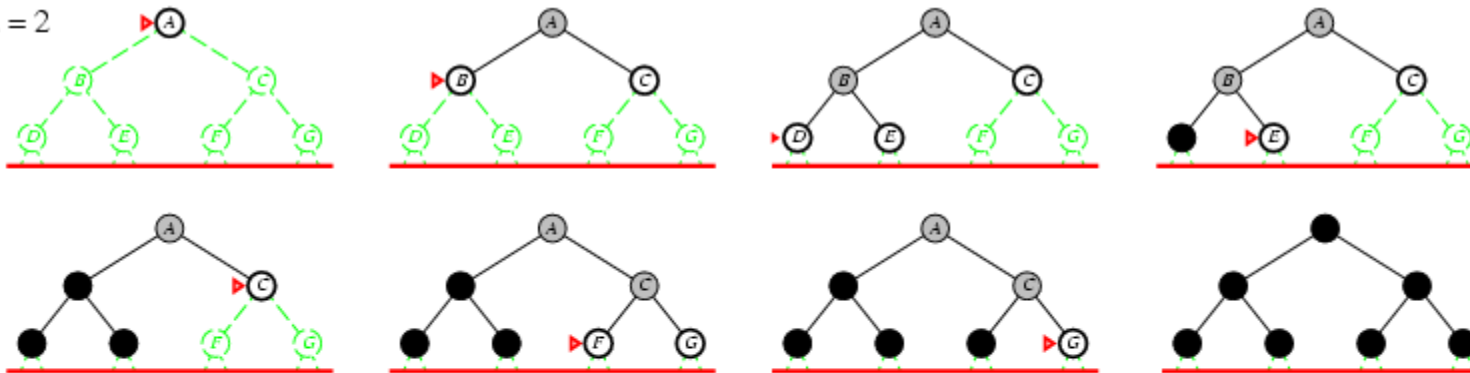
迭代加深搜索 $l = 1$

Limit = 1



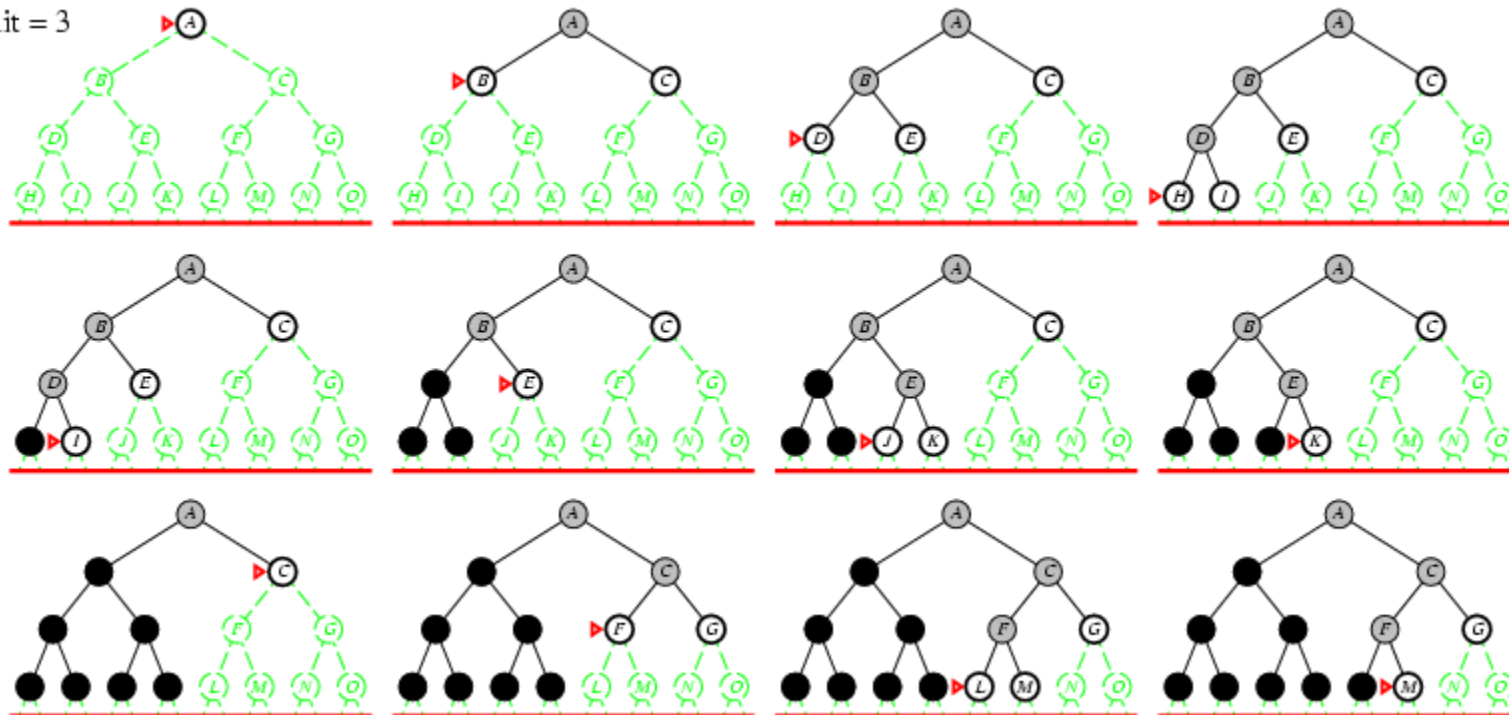
迭代加深搜索 $l = 2$

Limit = 2



迭代加深搜索 $l = 3$

Limit = 3



迭代加深搜索性质



- 时间复杂度（生成节点的个数）如果解在d层，则 $O(b^d)$
- 空间复杂度（存储节点的个数） $O(bd)$
- 完备性 是
- 最优性 是

迭代加深搜索性质

- 迭代深化搜索可能看起来很浪费，因为接近顶部的节点被重新生成多次
- 但是对于许多状态空间，大多数节点都在底层，所以上层的重复并不太重要。一般来说，大多数工作都发生在最低级别的搜索中，所以还不错！
- 设定深度为 s 时，

✓ 深度受限搜索生成的节点数：

$$N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{s-2} + b^{s-1} + b^s$$

✓ 迭代加深搜索生成的节点数：

$$N_{IDS} = (s+1)b^0 + sb^1 + (s-1)b^2 + \dots + 3b^{s-2} + 2b^{s-1} + 1b^s$$

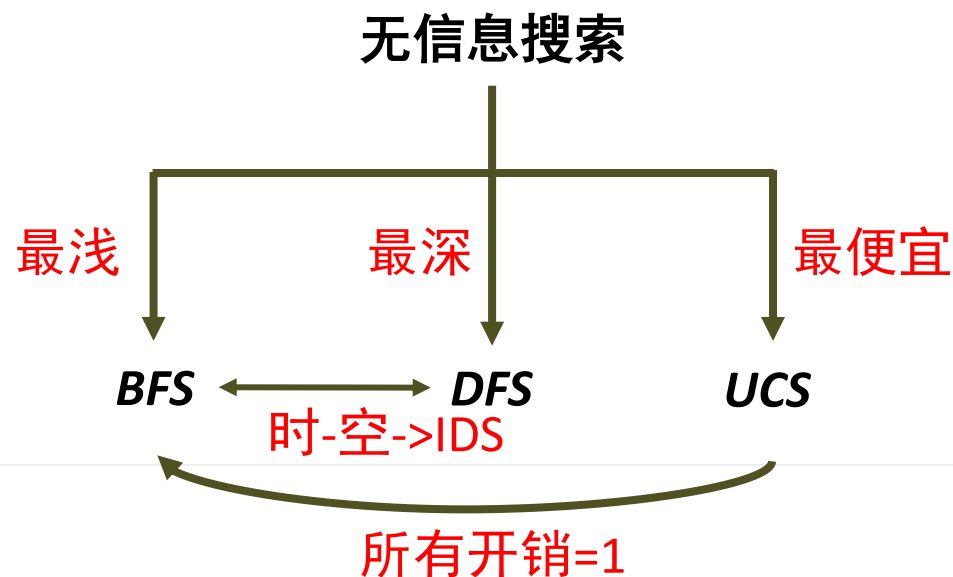
- 若 $b = 10, s = 5$,

$$✓ N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$$

$$✓ N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$$

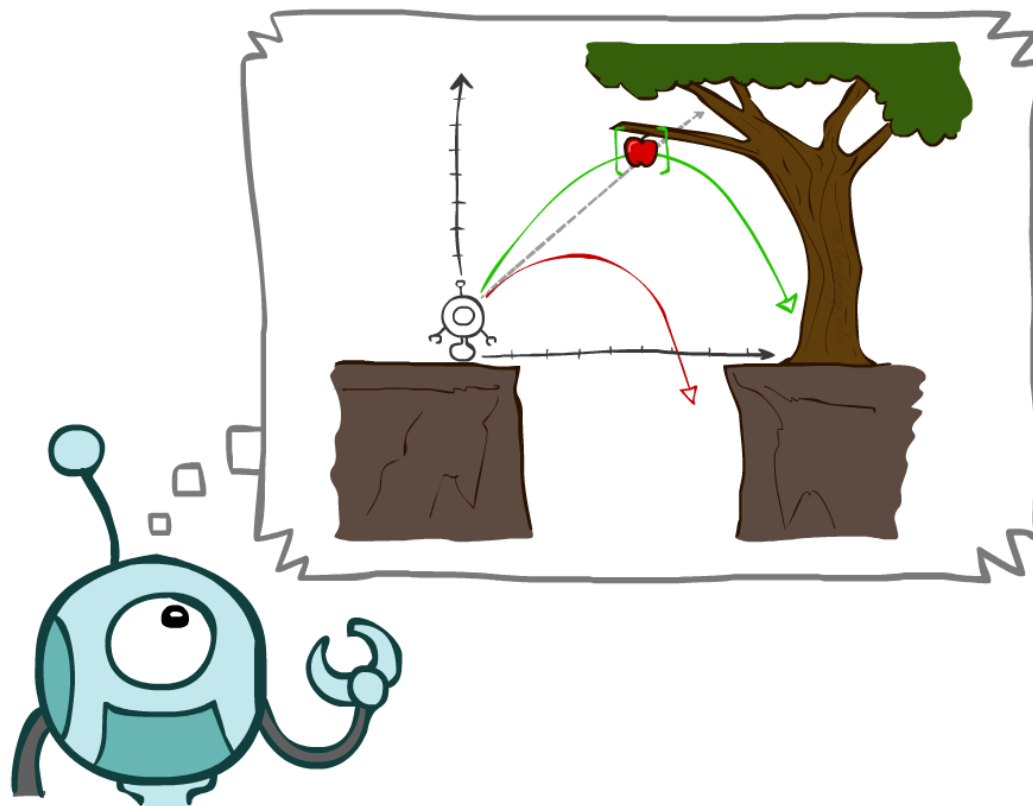
Review

评价标准	广度优先	代价一致	深度优先	深度有限	迭代加深
是否完备?	是	是	否	否	是
时间	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil + 1})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
空间	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil + 1})$	$O(bm)$	$O(bl)$	$O(bd)$
是否最优?	是	是	否	否	是



目录

- 搜索算法基础
- 无信息搜索（盲目搜索）
- 有信息搜索（启发式搜索）
- 局部搜索
- 约束满足问题



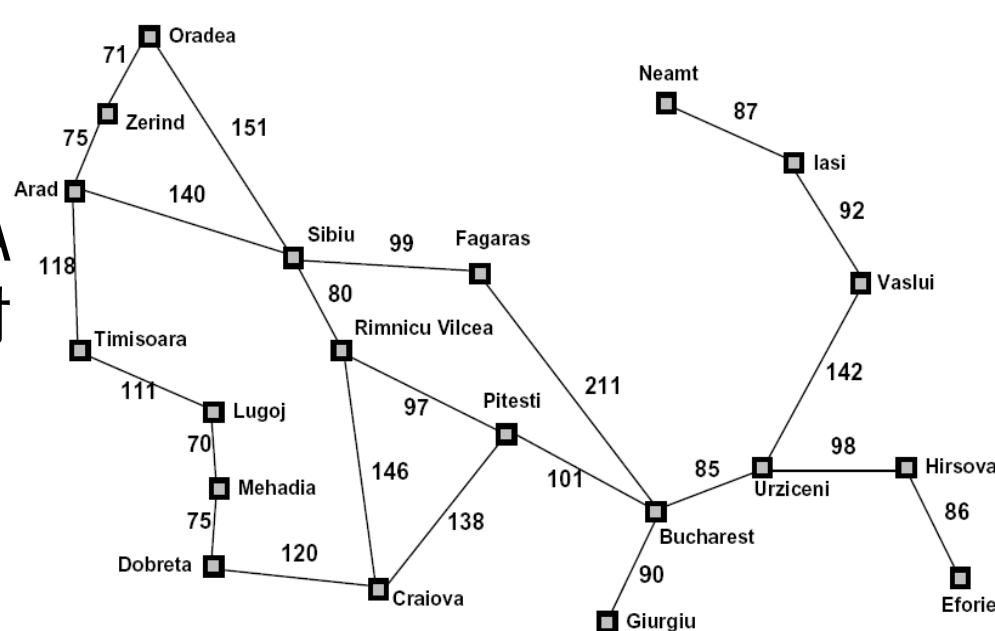
有信息搜索 vs 无信息搜索

- 无信息（盲目）搜索：对某个状态距离目标有多近毫无头绪
- 有信息（启发式）搜索：对某个状态距离目标有多近有一些辅助信息，比无信息策略更高效
- 其代表算法为贪心最佳优先搜索和A*搜索

启发函数与评价函数

辅助信息	所求问题之外、与所求解问题相关的特定信息或知识
评价函数 (evaluation function) $f(n)$	从当前节点 n 出发, 根据评价函数来选择后续访问节点
启发函数 (heuristic function) $h(n)$	计算从节点 n 到目标节点之间所形成路径的最小代价值

问题: 寻找从城市A到城市B之间行驶时间最短路线?



Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

辅助信息 (启发函数):
任意一个城市与终点城市B之间的直线距离

贪心最佳优先搜索

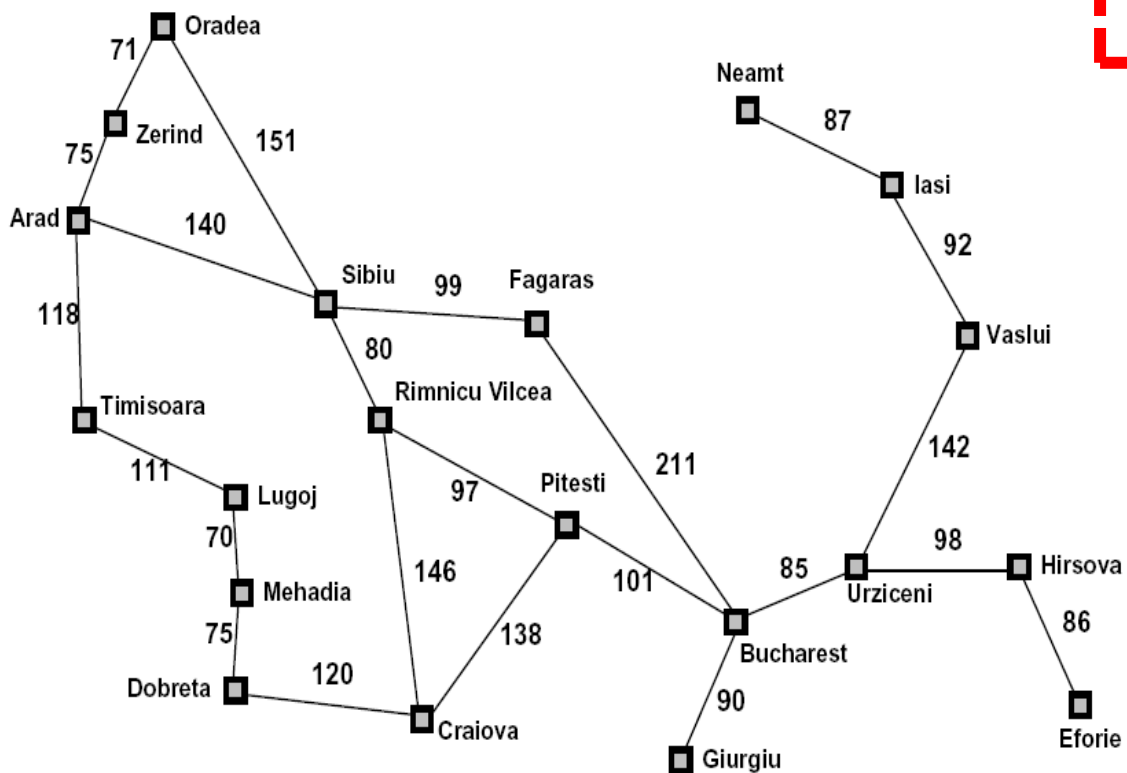
$f(n) = h(n)$: 首先访问**看起来**最接近目标的节点



贪心最佳优先搜索

$h(n)$

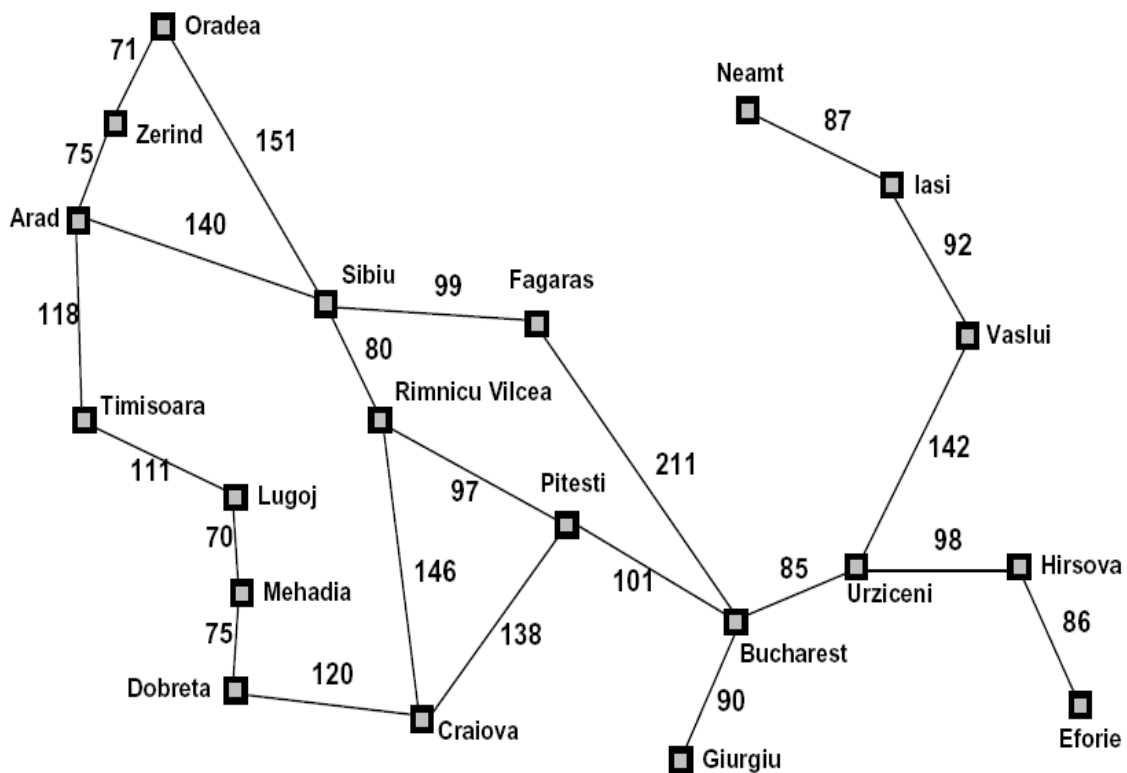
Arad
366



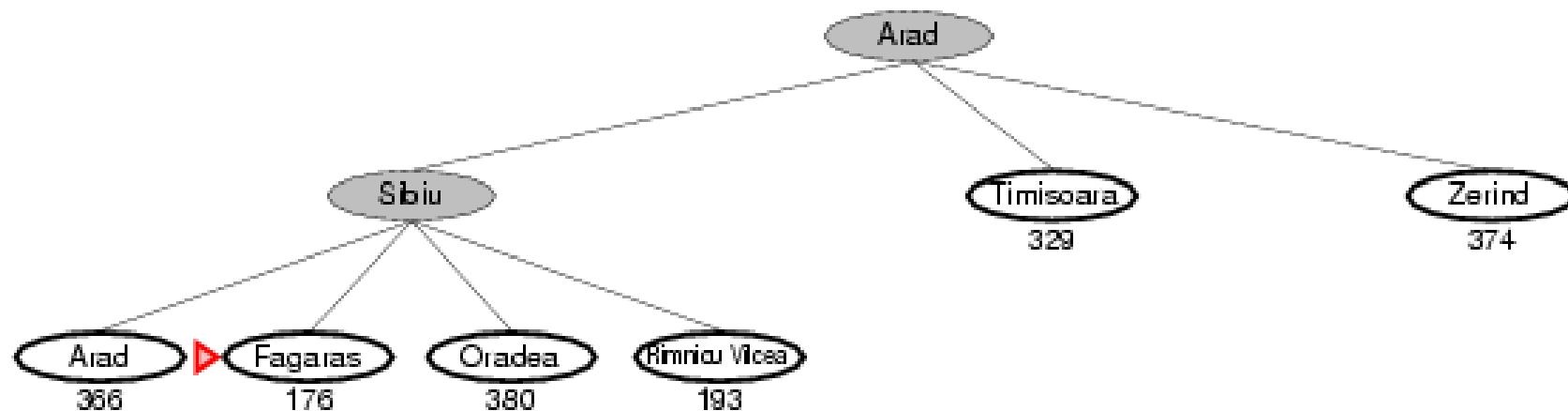
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

贪心最佳优先搜索

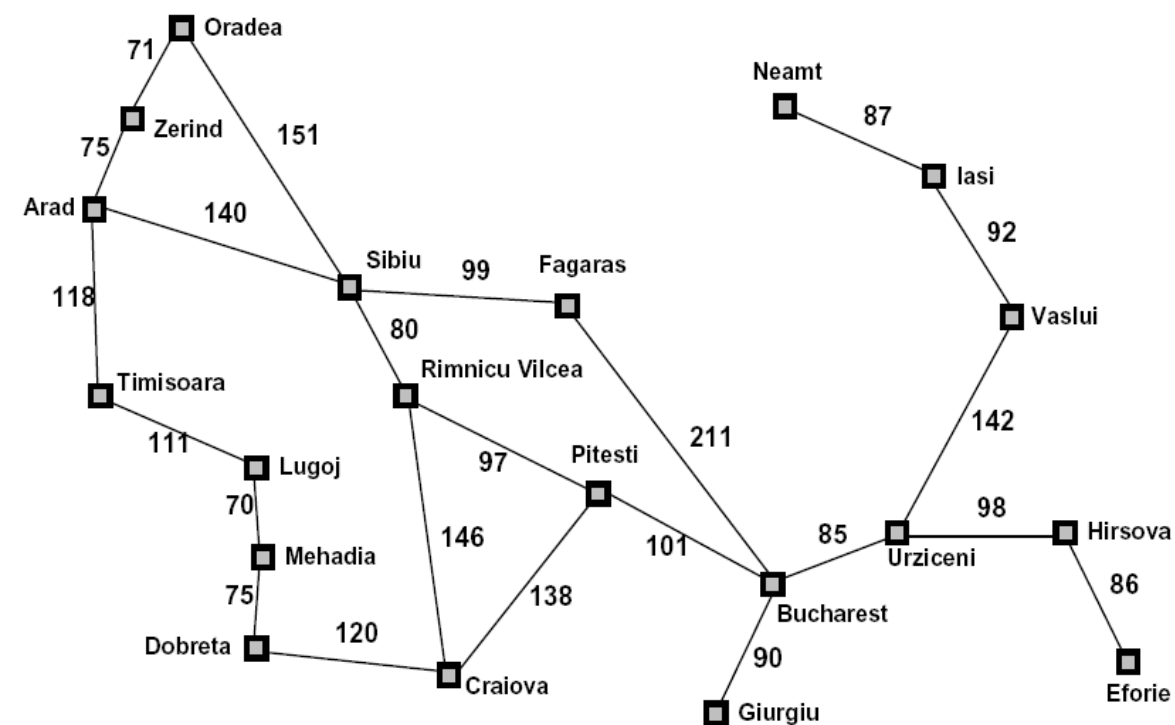


贪心最佳优先搜索

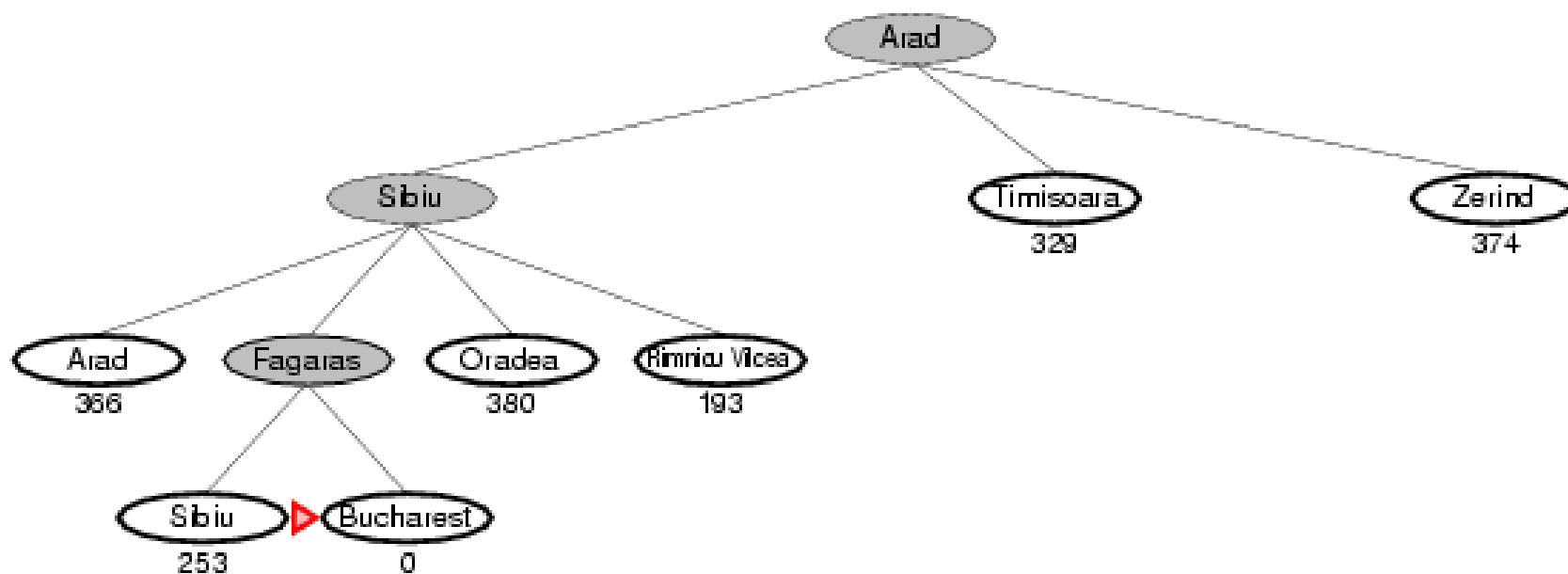


Straight-line distance
to Bucharest

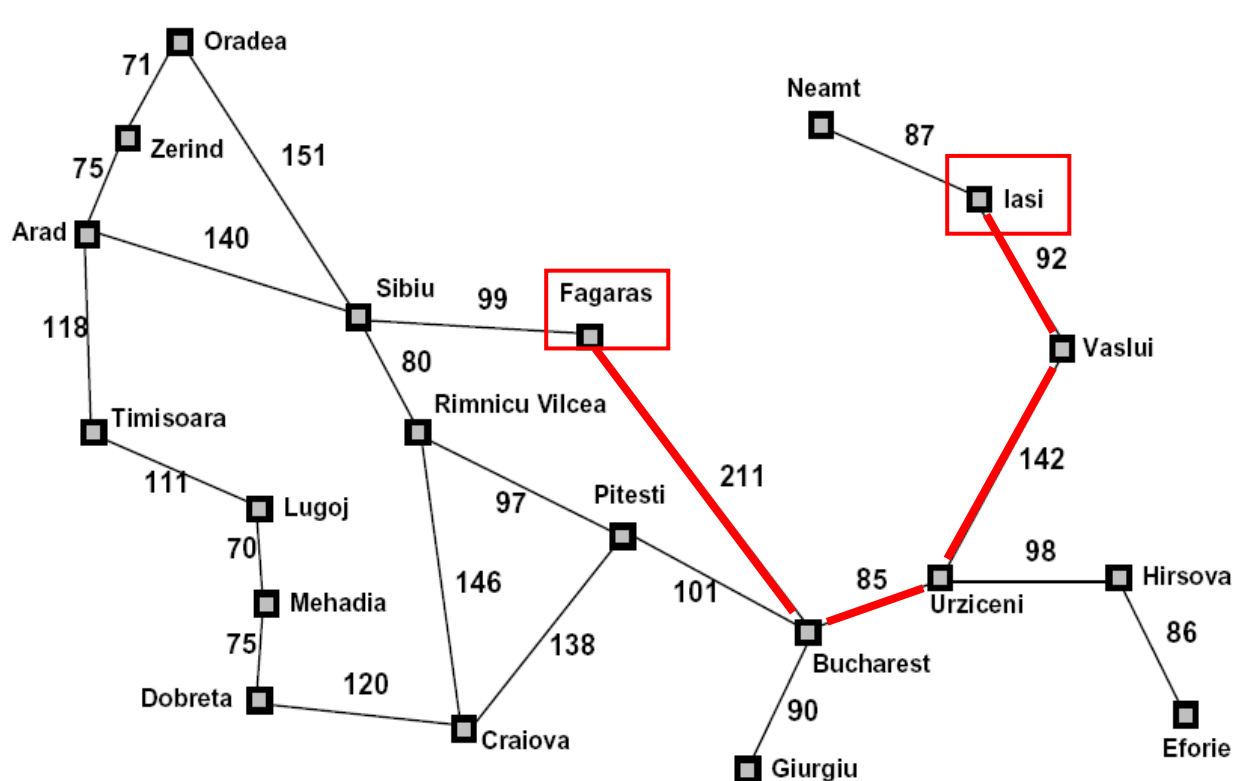
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



贪心最佳优先搜索



贪心最佳优先搜索问题



从Iasi->Fagara

- 贪心搜索会陷入循环：I-N
- 为什么会出错呢：没有考虑历史

贪心最佳优先搜索性质

- 时间复杂度（生成节点的个数） $O(b^{m+1})$ 类似深度优先搜索，会沿着一条路一直往下走，但是一个好的启发式可以带来显著的改进
- 空间复杂度（存储节点的个数） $O(b^{m+1})$
- 完备性 否，可能陷入循环
- 最优性 否

A* 搜索



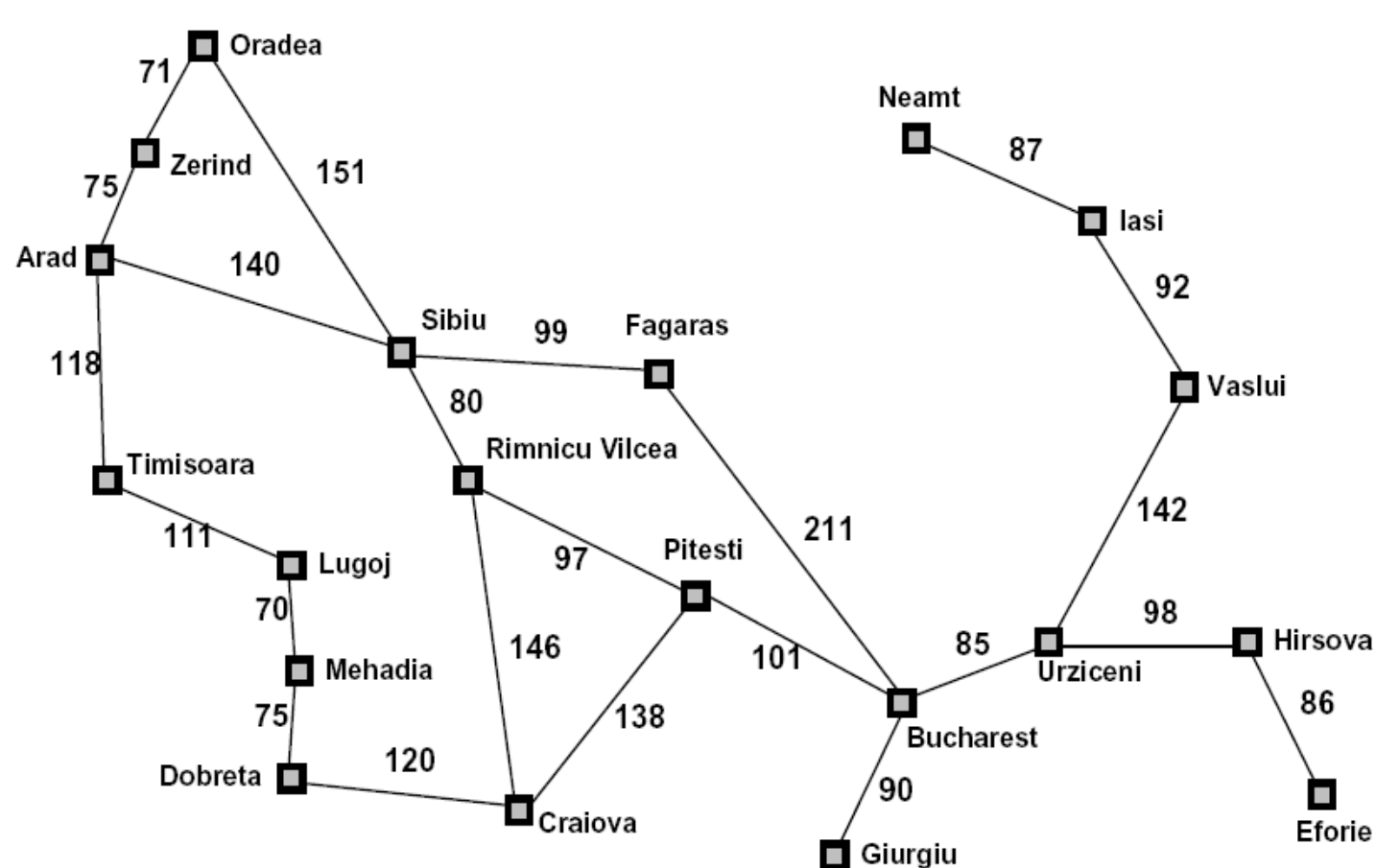
東南大學
SOUTHEAST UNIVERSITY



$f(n) = g(n) + h(n)$: 最小化总的估计代价

- $g(n)$: 从起始节点到节点 n 的开销（历史）
- $h(n)$: 表示从节点 n 到目标节点路径中所估算的最小开销（未来）
- $f(n)$: 可视为经过结点 n 、具有最小开销代价值的路径

A* 搜索

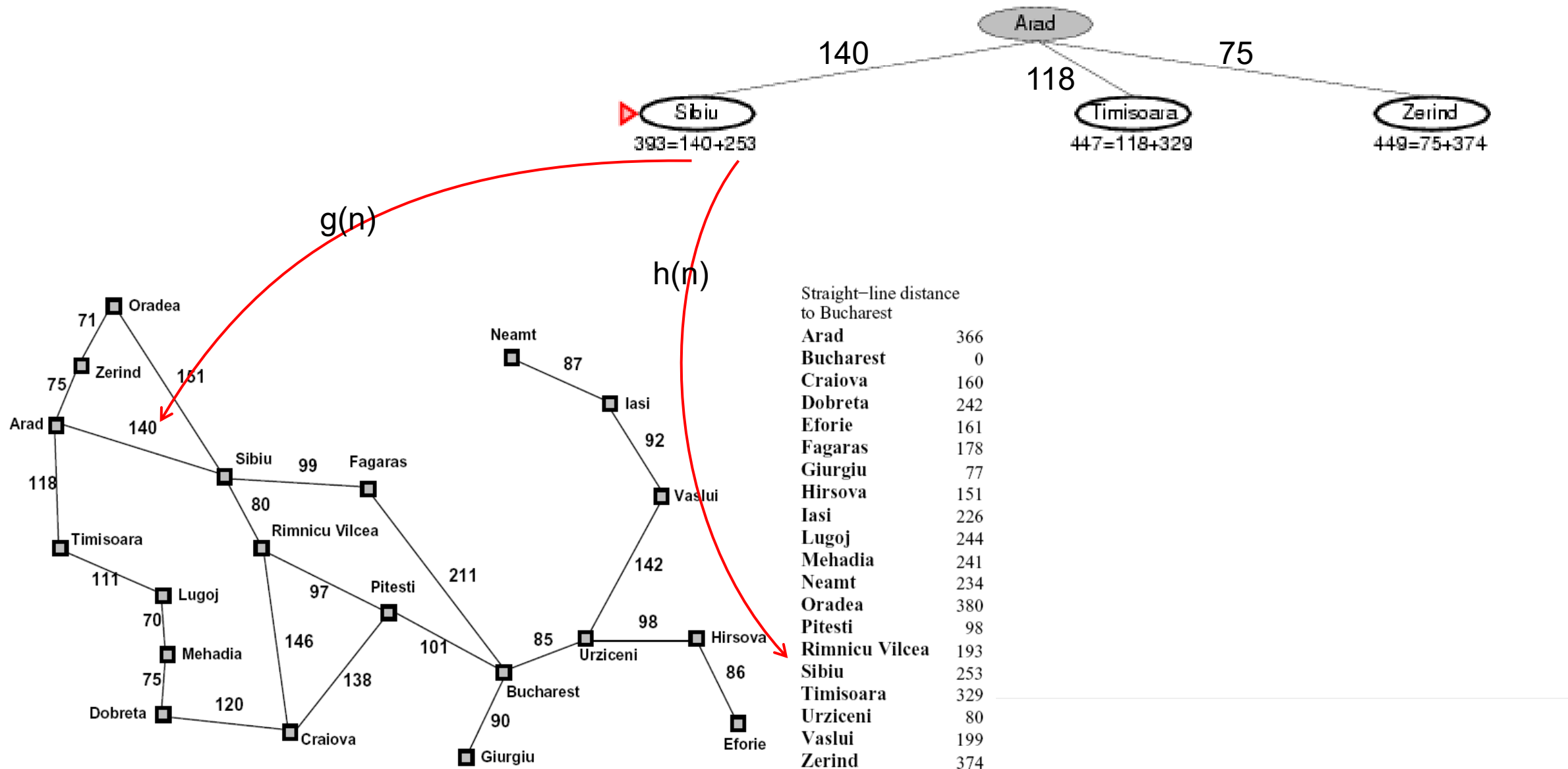


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

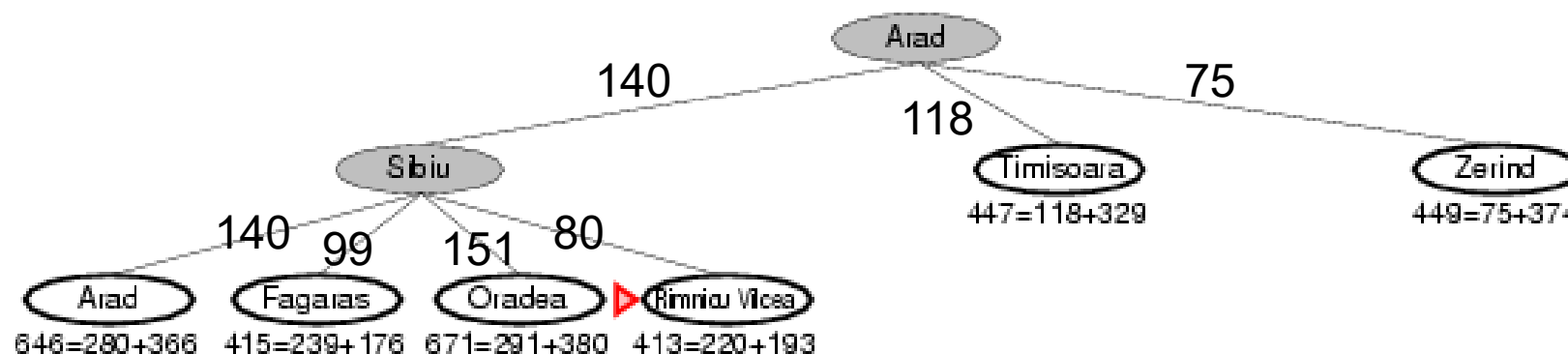
▶ **Arad**
366=0+366

A* 搜索



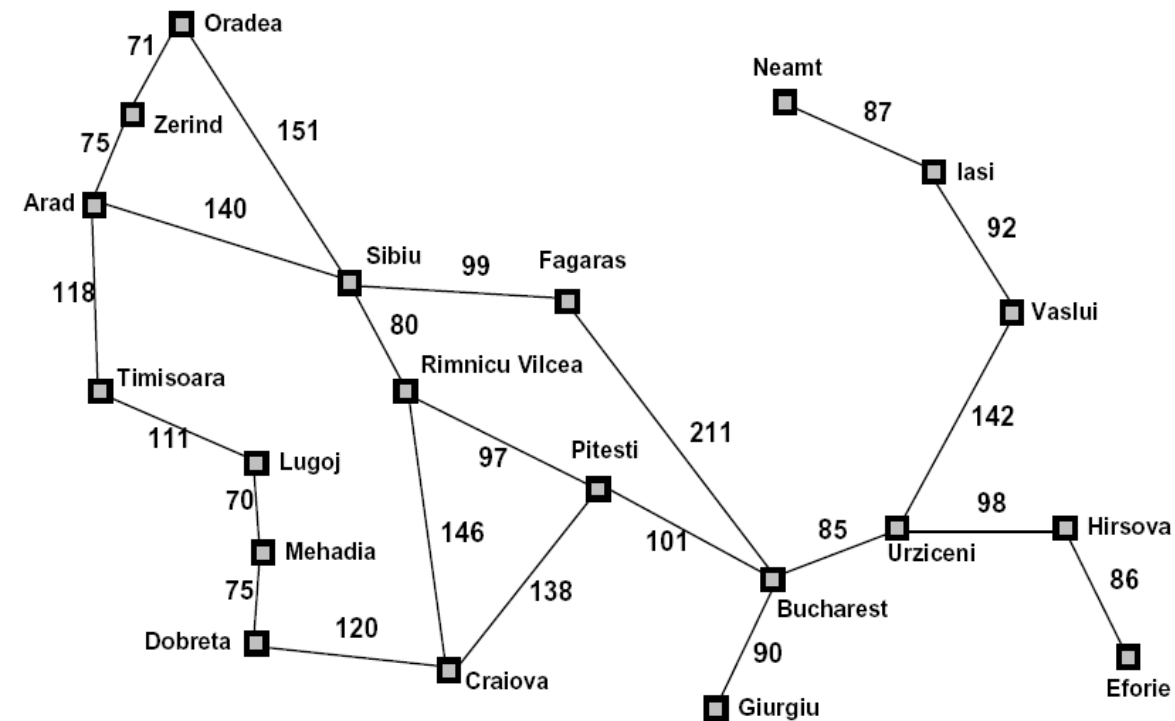


A* 搜索

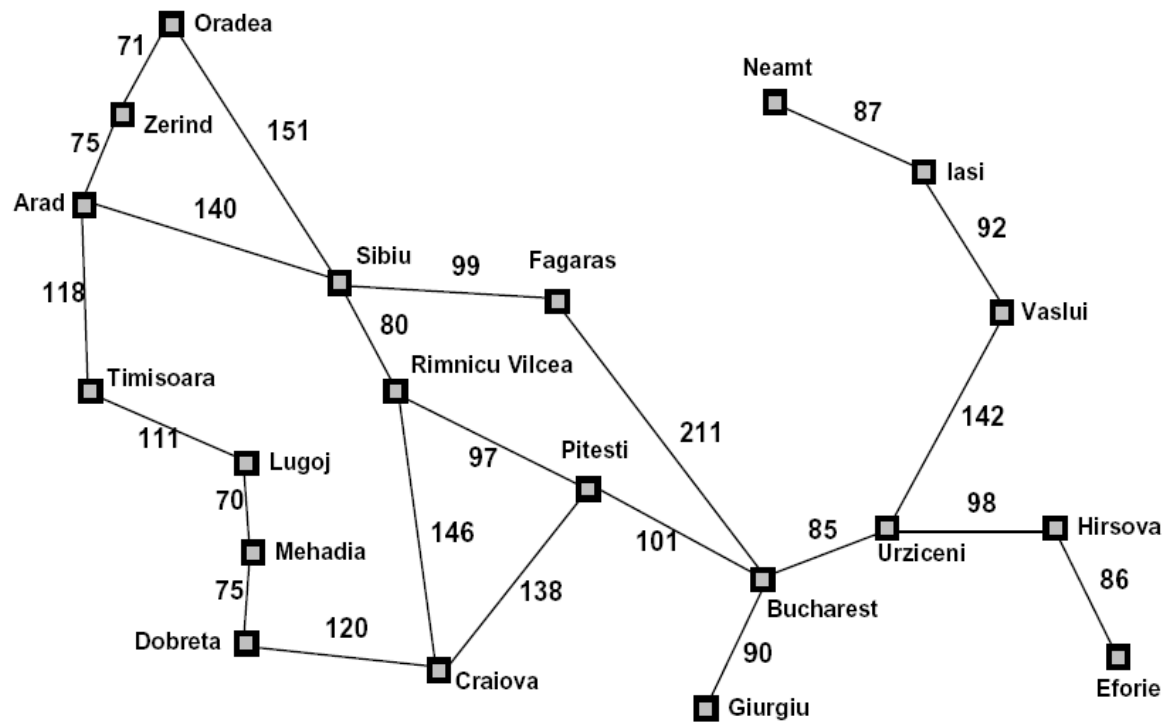
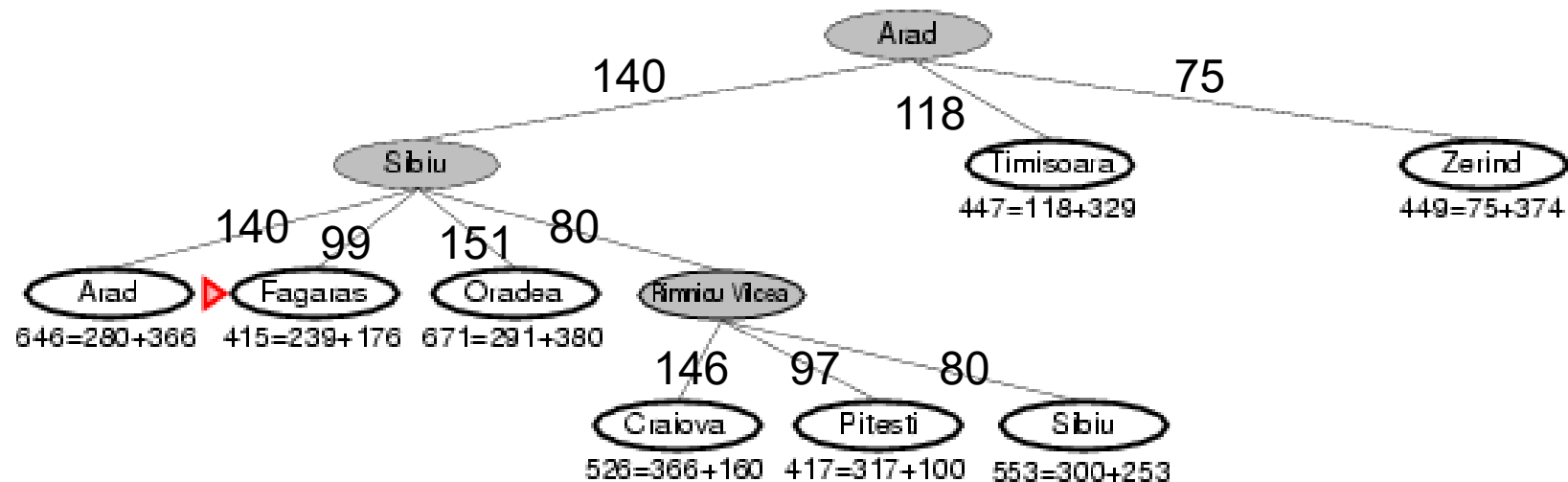


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



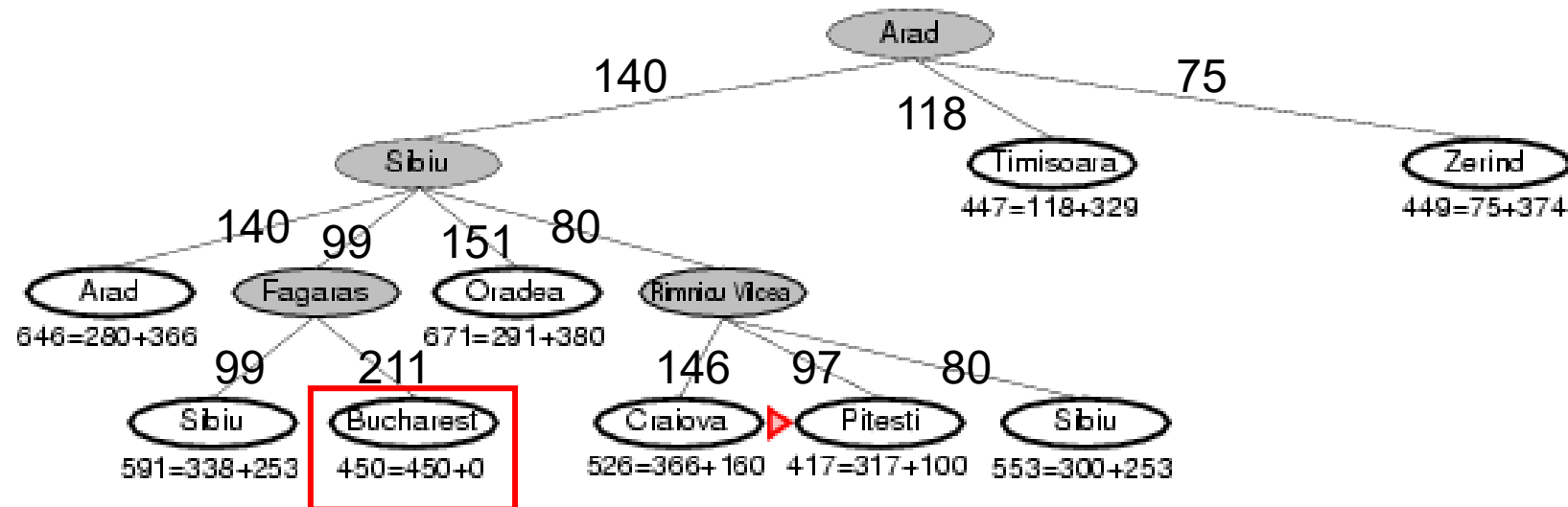
A* 搜索



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* 搜索

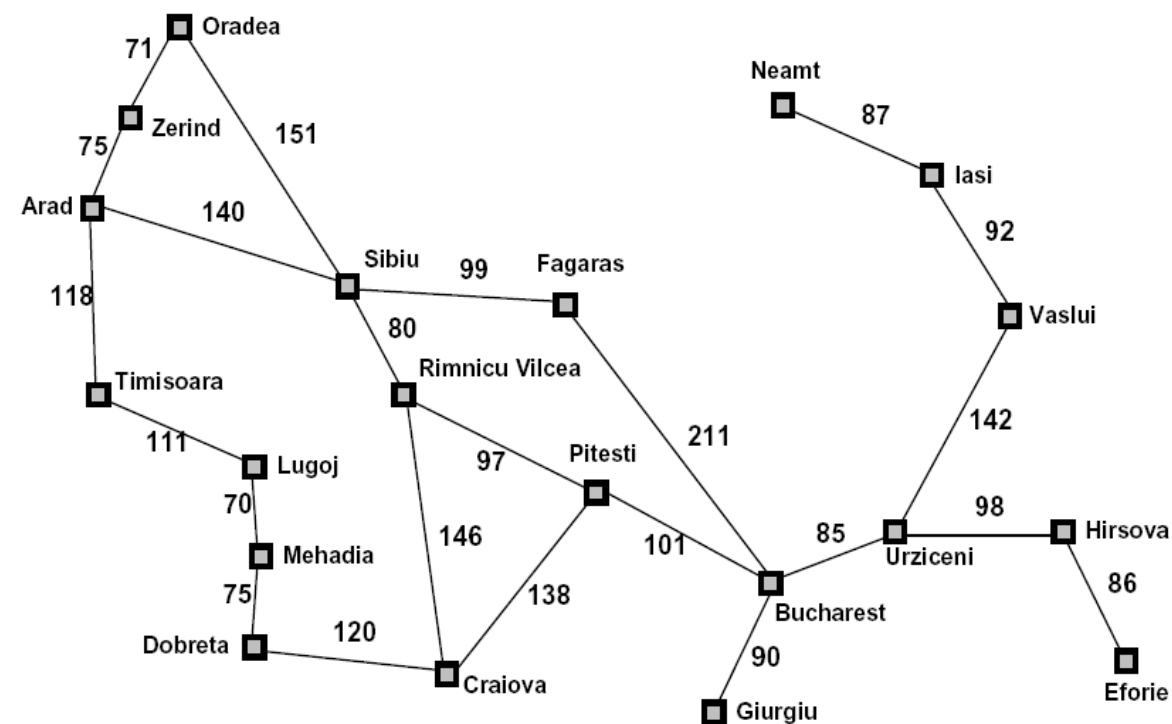


- 它不是边缘中代价最小的节点，可能存在一个经过Pitesti的解，代价低于417

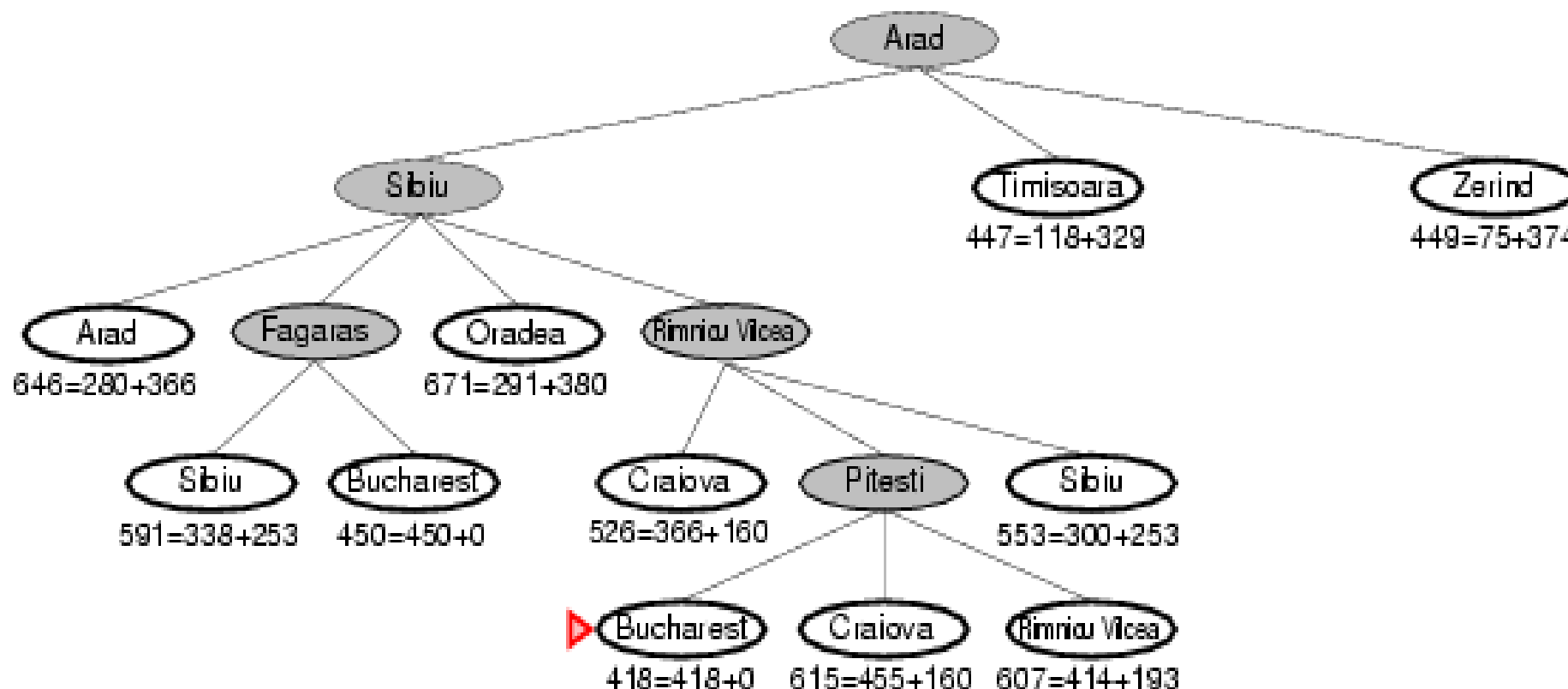
- 虽然已经发现，但此时算法没有选择它来访问，因此这是它没有被检测为一个解

Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

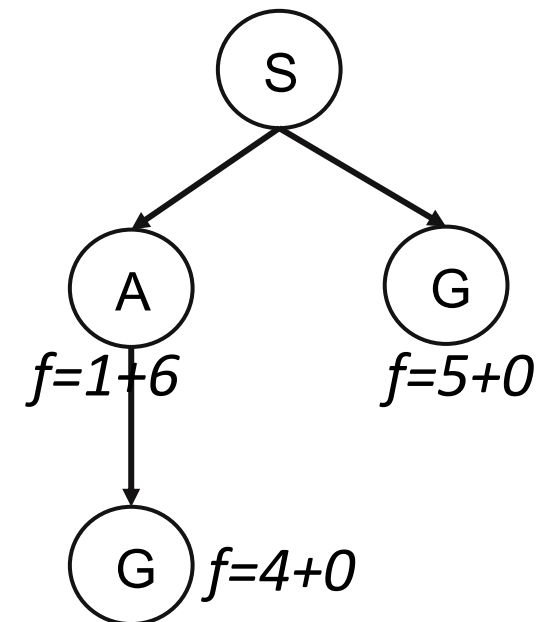
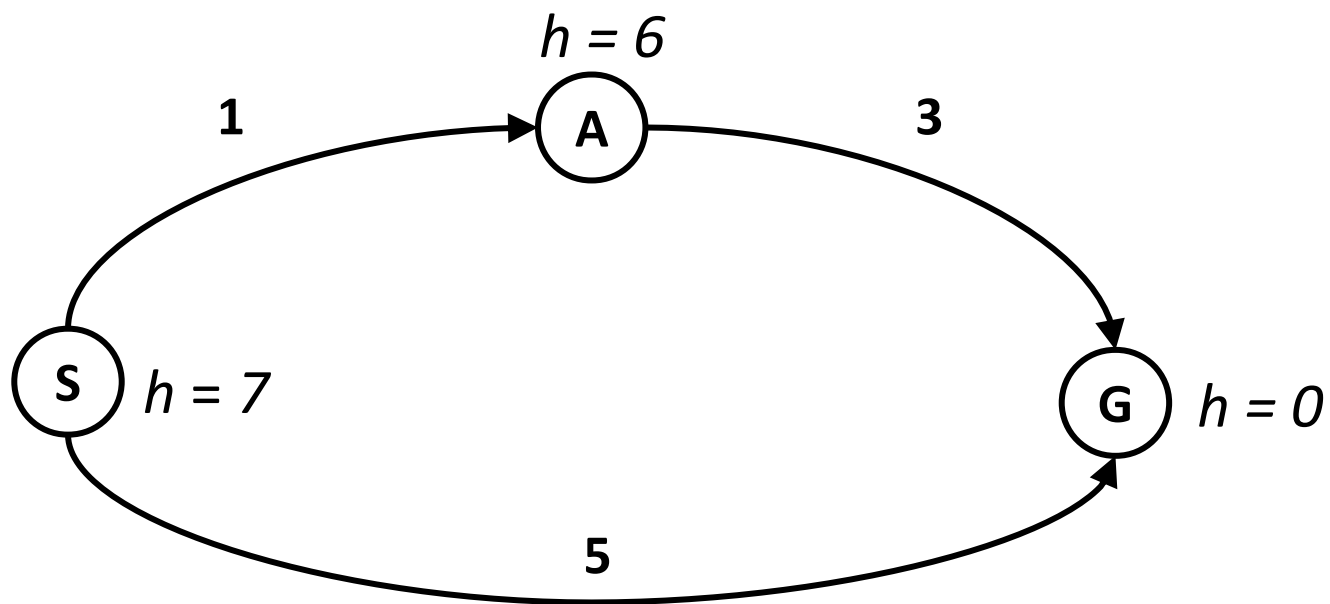


A* 搜索



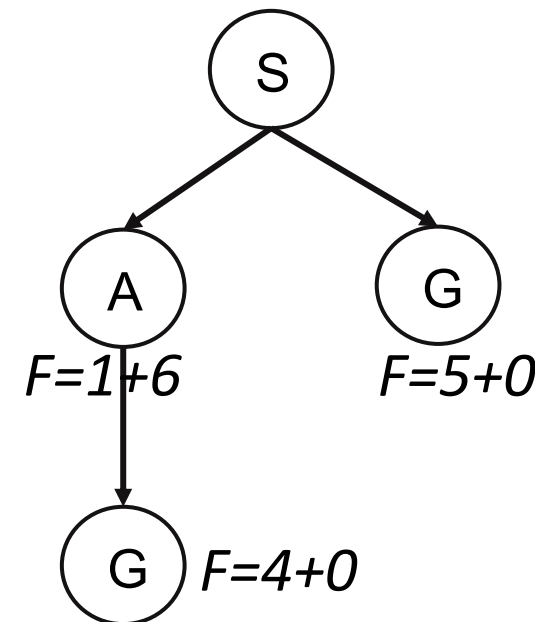
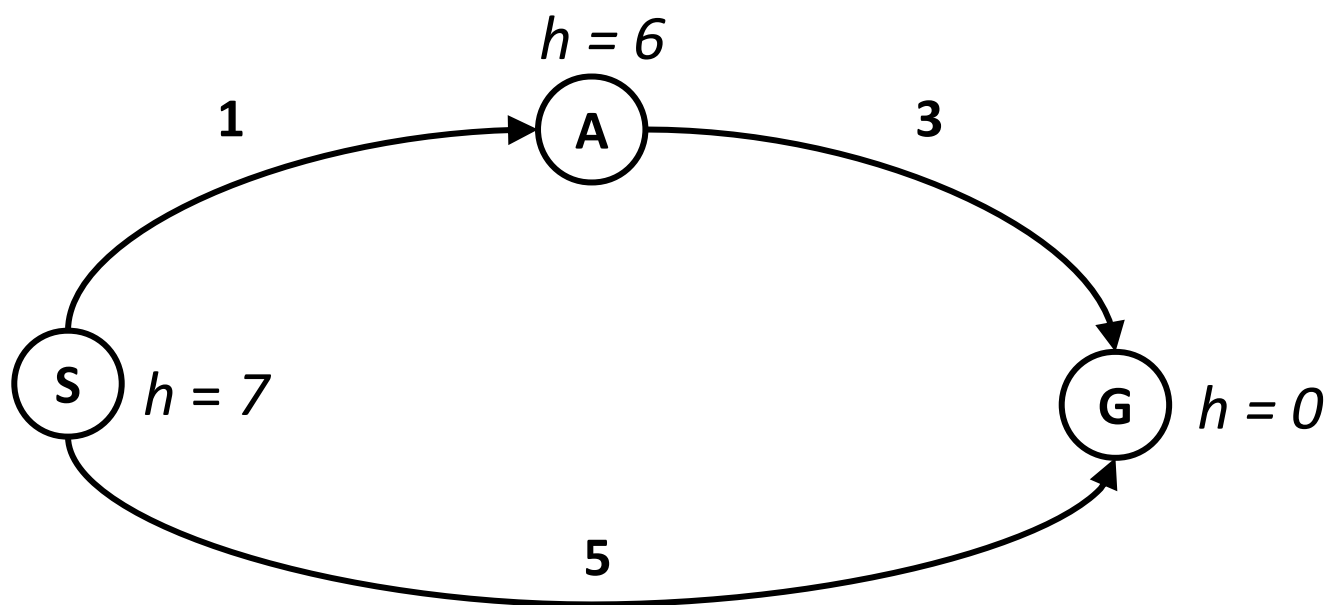
当目标状态是f值最低的节点时才停止搜索！

A* 搜索的最优性



- 一致代价: $S \rightarrow A \rightarrow G$
- A*: $S \rightarrow G$

A* 搜索的最优性



- 什么情况才能让A*找到最优路径?
- $f(A) < f(G)$, 即 $h(A) + g(A) < h(G) + g(G)$, 即 $h(A) < g(G) - g(A)$
- 即 从A到G的估计成本 < 从A到G的实际成本

可容许启发函数

可容许的：对于任意节点 n ，有 $h(n) \leq h^*(n)$ ，如果 n 是目标节点， $h(n)=0$

- $h^*(n)$ 是节点 n 到达目标节点的真实开销
- 一个可容许的启发函数意味估计代价小于等于实际代价，即，它是乐观的
- 比如 $h(n) = \text{直线距离}$
- 定理：若 $h(n)$ 是可容许的，则**树搜索 A* 算法**是最优的

证明最优性

- 假设：A是最优的目标节点,B是次优的目标节点, h 是可容许的
- 要证：A将在B之前访问，即，A比B更早从边缘弹出

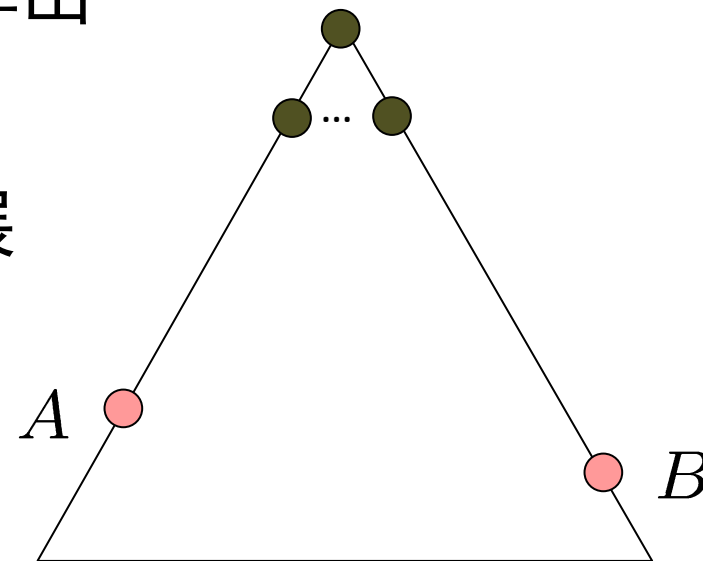
- 情况1：A和B同时处于边缘，要证A将被优先扩展
 - ✓ 即要证A具有更小的 f 值

$$f(A) = g(A) + h(A) = g(A)$$

$$f(B) = g(B) + h(B) = g(B)$$

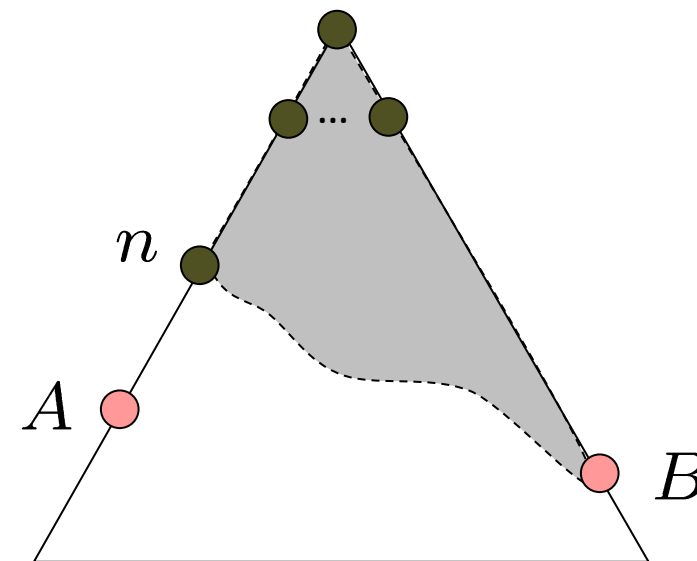
- ✓ A是最优目标节点，说明A的实际代价比B小，即 $g(A) < g(B)$

- ✓ 因此 $f(A) < f(B)$

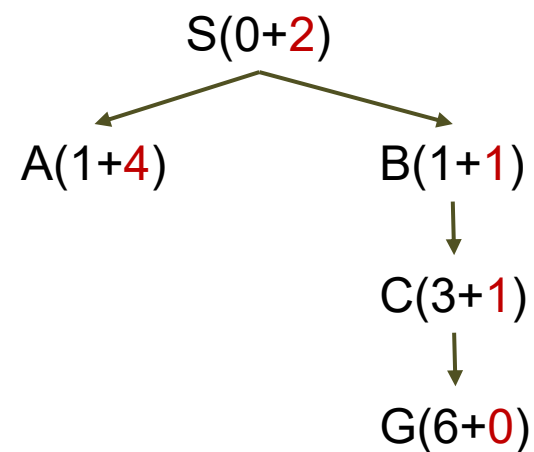
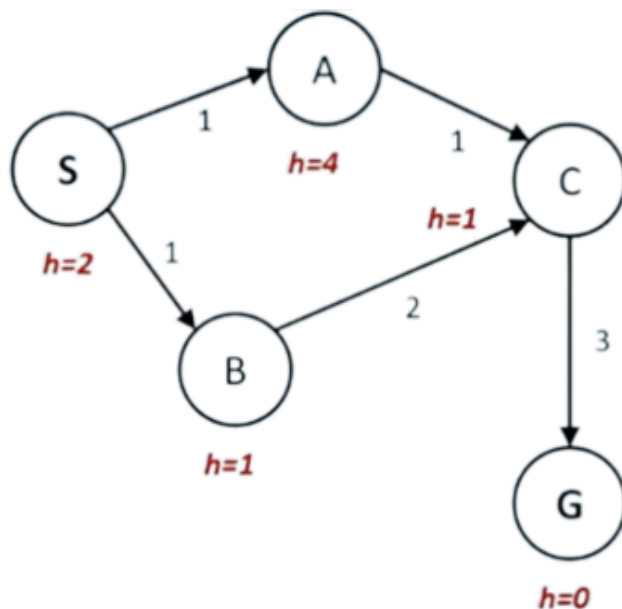


证明最优性

- 情况2: B在A之前进入边缘
 - ✓ 则要先证明A的**所有祖先节点n**都在B之前访问, 否则A还没进入边缘, B已经被访问了, 即 $f(n) \leq f(B)$
 - ✓ 因为 h 是可容许的, 对n到A的估计开销要小于实际开销 $h(n) \leq g(n \rightarrow A)$
 - ✓ 那么 $\underbrace{[g(n) + h(n)]}_{f(n)} \leq \underbrace{[g(n) + g(n \rightarrow A)]}_{g(A) = f(A)}$
 - ✓ 因为 $f(A) < f(B)$, 有 $f(n) \leq f(A) < f(B)$
 - ✓ 即n比B先被访问



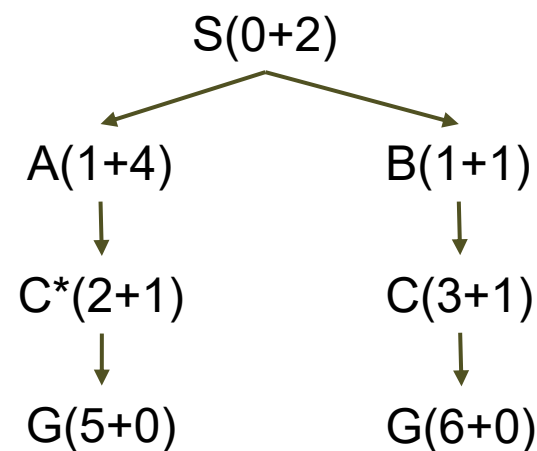
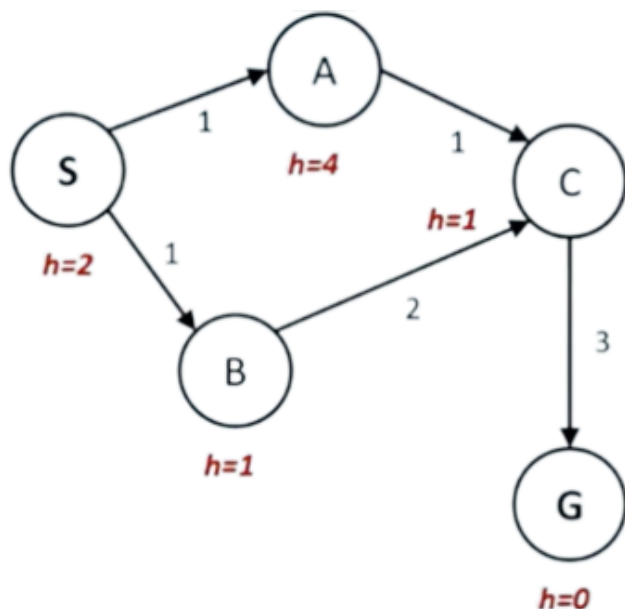
图搜索中的最优性



- 最优: $S \rightarrow A \rightarrow C \rightarrow G$
- A^* : $S \rightarrow B \rightarrow C \rightarrow G$

步骤	边缘	扩展节点	生成节点	已搜索集合
1	S(2)	S	A, B	-
2	A(5), B(2)	B	C	S
3	A(5), C(4)	C	G	S, B
4	A(5), G(6)	A	C* (skip)	S, B, C
5	G(6)	G	S, A, B, C	S, A, B, C

图搜索中的最优性



- 需要在第一次访问节点C的时候，就保证路径的最优性！
 - ✓ C*需要在C之前被扩展
 - ✓ C*的祖先节点需要在C之前被扩展
 - ✓ $f(A) < f(C)$ ，即 $g(A) + h(A) < h(C) + g(C)$ ，即 $h(A) - h(C) < g(C) - g(A)$
 - ✓ 对于搜索图中**每一条边**的估计成本都要小于等于实际成本

一致性启发函数

一致的：把任何一个点当做目标，启发式函数都是可容许的

定理：若 $h(n)$ 是一致的，则**图搜索 A* 算法**是最优的

A* 搜索性质

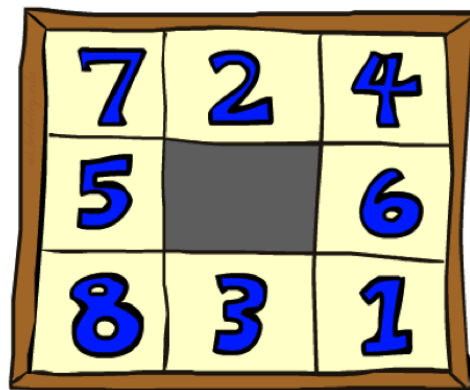
- 时间复杂度（生成节点的个数） 指数级，如果启发式函数选的很差，会退化成类似广度优先搜索，像涟漪一样均匀地向外搜索
- 空间复杂度（存储节点的个数） 指数级，类似广度优先搜索
- 完备性 是
- 最优性 是，如果启发式函数是可容许的/一致的

启发式函数的设计

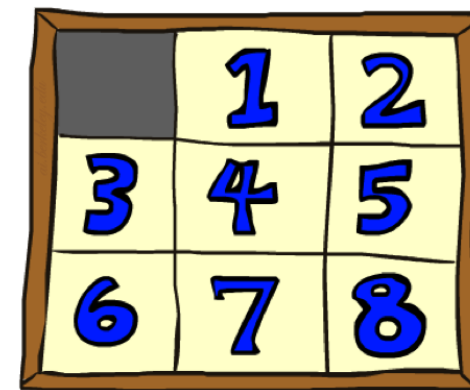


- **松弛问题**：在每个状态下的真实代价都小于或者等于原问题的代价
- 找到原始问题的松弛问题后，该松弛问题的解可以作为原问题的可容许启发式函数值

启发式函数的设计



- 松弛问题1: 每个方块上可以叠加多个数字
- 启发式函数: 曼哈顿距离
- $h(\text{start}) = 3+1+2+\dots=18$



- 松弛问题2: 可以把数字取下
- 启发式函数: 摆错位置的数字的数量
- $h(\text{start}) = 8$

启发式函数的质量



- 在现实中， A^* 的搜索树有的地方深，有的地方浅，这很难直观判断启发式函数好不好
- 描述启发式质量的一种方法是**有效分支因子** b^* ：假设有一棵各层分支数都完全一样的标准平衡树，如果它的深度是 A^* 搜索的步数 d ，且节点总数等于 A^* 生成的节点数 N ，那么这棵标准树的分支数是：

$$N + 1 = 1 + b^* + \dots + (b^*)^d$$

- 那么，可以用 b^* 衡量 A^* 的效率。 b^* 越接近 1，说明 A^* 越快， b^* 越大说明生成的无效节点越多

启发式函数的质量



- 如果对所有节点，都有 $h_2(n) \geq h_1(n)$ 且它们都是可容许的，那么我们说 h_2 dominates 占优于 h_1
- 优势可以直接转化为效率：使用 h_2 的 A* 永远不会比使用 h_1 的 A* 扩展更多的节点